GAPS_AND_RISKS_REGISTER

GAPS_AND_RISKS_REGISTER — HelixKnowledge Skill Graph System

Revision: 10 **Last modified:** 2026-07-18T06:30:00Z

> Adversarial audit satisfying operator mandate **R17**. Every row carries concrete `file:line` evidence (positive-evidence-only, R11). Anything not directly verified is labelled **UNCONFIRMED**. Audit date: 2026-07-15. Scope audited: `IMPLEMENTATION_PLAN.md`, `REQUIREMENTS.md`, `SPEC.md`, `api/openapi.yaml`, and the full `project/` Go backend (`github.com/helixdevelopment/skill-system`, 53 `.go` files, 0 tests).
>
> **Method note:** `go build`/`go vet` are reported green by the task; this audit did not re-run them. Findings are about *design, behaviour, wiring, security, and contract fidelity*, not compilation.

Summary counts (2026-07-18 Rev 10 — all items G01–G137)

Status	Count	IDs
OPEN — CRITICAL	1	G04
OPEN — HIGH	64	G09, G10, G12, G14, G15, G40, G42, G43, G59, G63, G69–G92 (×24), G93–G122 (×30)
OPEN — MEDIUM	25	G17, G18, G30, G44, G45, G47, G55, G56, G58, G60, G61, G66,

Status	Count	IDs
		G123, G124–G135 (×12)
OPEN — LOW	4	G37, G62, G67, G68
FIXED	41	G01, G02, G03, G05, G06, G07, G08, G11, G13, G16, G19, G20, G21, G22, G23, G24, G25, G26, G27, G28, G29, G31, G32, G33, G34, G35, G36, G38, G39, G41, G46, G48, G49, G51, G52, G53, G54, G57, G64, G65, G137
OPEN — CRITICAL	3	G01, G03, G04
OPEN — HIGH	63	G09, G10, G14, G15, G40, G42, G43, G59, G63, G69–G92 (×24), G93–G122 (×30)
OPEN — MEDIUM	25	G17, G18, G30, G44, G45, G47, G55, G56, G58, G60, G61, G66, G123, G124–G135 (×12)
OPEN — LOW	4	G37, G62, G67, G68
FIXED	40	G02, G05, G06, G07, G08, G11, G12, G13, G16, G19, G20, G21, G22, G23, G24, G25, G26, G27, G28, G29, G31, G32, G33, G34, G35, G36, G38, G39, G41, G46, G48, G49, G51, G52, G53, G54, G57, G64, G65, G137
OPEN — HIGH	63	G09, G10, G12, G14, G15, G40, G42, G43, G63, G69–G92 (×24), G93–G122 (×30)
OPEN — MEDIUM	24	

Status	Count	IDs
		G17, G18, G30, G44, G45, G47, G55, G56, G58, G61, G66, G123, G124–G135 (×12)
OPEN — LOW	3	G37, G67, G68
FIXED	43	G02, G03, G05, G06, G07, G08, G11, G13, G16, G19, G20, G21, G22, G23, G24, G25, G26, G27, G28, G29, G31, G32, G33, G34, G35, G36, G38, G39, G41, G46, G48, G49, G51, G52, G53, G54, G57, G59, G60, G62, G64, G65, G137
N/A	1	G136 (meta-assessment task itself)
TOTAL	**136**	(G01–G135 + G137; G136 is the assessment task, deliberately unrated)

> **Register-cleanup note (2026-07-17 Rev 8):** Field-verification pass against every per-item STATUS line. Corrections from Rev 7: - G43 was wrongly in FIXED row; moved to OPEN HIGH (per-item status: FILED, PENDING). - G19 (Fixed → SPEC.md), G20 (COMPLETED), G28 (Implemented) were in OPEN MEDIUM but have Fixed per-item statuses; moved to FIXED. - G137 was in OPEN HIGH but has Fixed per-item status; moved to FIXED. - FIXED count corrected from 75 → 39 (Rev 7 count was inflated; only 39 items have Fixed/Implemented/Completed/CLOSED per-item STATUS lines). - OPEN HIGH corrected from 28 → 64 (G43 added; G137 removed; range counts 24+30 = 54 from G69–G122 are correct). - OPEN MEDIUM corrected from 26 → 25 (G19, G20, G28 removed; G28 was missed in Rev 7's count adjustment). - G58 remains a placeholder (Type: TBD, Status: UNCONFIRMED) — no real finding filed; conductor must investigate and create the actual finding. The Fixed row captures all items whose per-item

> STATUS line reads Fixed, Implemented, Completed, or CLOSED. Open-item severity rows contain only OPEN items. **Open-total verification:** 3+63+25+4 = 95 open + 40 fixed + 1 N/A = 136 total. ✓

> Severities for G52–G137 are **proposed** per G136 — see `research/g136_severity_assessment.md` for the per-item evidence and rationale. These proposals supersede the 2026-07-16 "not yet folded" note and are now the working baseline for the register. Items marked CLOSED remain in the historical counts but carry explicit `(closed)` annotations in their status lines.

Headline: the running binary is not the audited/hardened codebase

The single most important structural finding is that **`cmd/server/main.go` runs its own ad-hoc API and never imports `internal/api`, `internal/validation`, or `internal/autoexpand`.** The hardened, spec-shaped handlers (with API-key auth, strict CORS, the "zero-bluff" jury pipeline, and the auto-growth pipeline) all exist as source but are **dead code / unwired**. The R1 security fixes and the R2/R8/R11 flagship features are therefore present in the tree but absent from the artifact — a textbook §11.4.108 SOURCE≠RUNTIME gap. G01/G02/G03 all flow from this.

CRITICAL

G01 — Two rival API servers; the hardened `internal/api` is unwired dead code, the live server has no auth + wildcard CORS

- **Category:** inconsistency / security
- **Severity:** critical — the R1-mandated CORS + api_key fixes are *not in the running binary*; the live REST surface is unauthenticated.
- **Evidence:**
  - Hardened server exists: `internal/api/server.go:82-210` (`Server`, `APIKeyAuth`, strict `CORS`), `internal/api/middleware.go:280-310` (`APIKeyAuth`), `internal/api/middleware.go:328-387` (allowlist `CORS`), `internal/api/skills_handler.go` (full CRUD).
  - `internal/api` has **zero importers** (`grep -rln skill-system/internal/api` → no match).
  - The actually-run server is `cmd/server/main.go:140-283` `setupAPI()`, which builds a *second* Gin router with its own `corsMiddleware()` that sets `Access-Control-Allow-Origin: *`

unconditionally (`cmd/server/main.go:362-373`) and applies **no API-key auth** anywhere.
    - Even the hardened path is fail-open: auth is applied only `if len(s.cfg.APIKeys) > 0` (`internal/api/server.go:163`) — empty key set ⇒ all `/api/v1` open.
- **Why it matters:** OpenAPI declares `ApiKeyAuth` on every endpoint; the deployed server enforces none. Anyone who can reach the port can read/write. It also means the "security clean" P0.T2 gate is satisfiable only against dead code — an anti-bluff (R11) hazard.
- **DECISION:** Delete the `setupAPI()` router in `cmd/server/main.go`; wire `internal/api.Server` as the single REST surface, constructed with a real `Pool` adapter. Make auth **fail-closed**: if no keys configured, refuse to start (or bind loopback-only) rather than serve open. **Alternatives rejected:** (a) keep both and "pick at runtime" — guarantees drift and was the exact R1 dedupe smell; (b) add auth to the ad-hoc router — duplicates the hardened logic a third time.
- **STATUS (2026-07-18) — FIXED:** Runtime security hole closed (2026-07-15, Fable-xhigh GO). Dead-server consolidation completed (2026-07-18): removed 6 dead handler files (`skills_handler.go`, `expand_handler.go`, `learn_handler.go`, `registry_handler.go`, `search_handler.go`, `server.go`) + dead `*Server` methods from `system_handler.go` — total -1826 lines. Alive standalone functions (`parseRequestBody`, `convertTOMLWrapper`, `exportToTOMLWrapper`, `MetricsHandler`, `VersionHandler`, etc.) extracted to `request_helpers.go`. `internal/api` coverage improved from 41.6% to 59.8%. Per §11.4.124: verified via grep that `Server` had zero non-test callers; all handler methods were only wired in dead `Server.RegisterHandlers()`. Full test suite 27/27 packages GREEN.
- **Test coverage:** integration (auth 401 on missing/invalid key, fail-closed on empty key set), security (disallowed Origin gets no ACAO header; wildcard never co-occurs with credentials), contract (routes == OpenAPI), regression, smoke, mutation (reintroduce reflect-origin / drop auth → test fails). **Challenges:** yes (end-to-end unauthorised-access probe). **HelixQA:** yes.

G02 — The "sandbox" provides no isolation: default path executes arbitrary skill code on the host (RCE), with false security claims

- **Category:** security / danger-zone
- **Severity:** critical — latent (see reachability) but a full remote-code-execution primitive by design.

- **Evidence:**
  - Default `sandbox_type = "wasm"` (`internal/config/config.go:179`, `config/config.toml`), and `createDefaultSandbox` maps `wasm` → `NewWASMSandbox` (`internal/validation/pipeline.go:121-135`).
  - When no WASM runtime is found (the default on a bare host) `Execute` falls through to `executeProcess`, which runs code via the host interpreter: `python -c`, `bash -c`, `node -e` (`internal/validation/sandbox.go:86-105`, `206-290`, dispatch `237-245`). The Go path is literally `go run` on the host (`sandbox.go:143-203`).
  - Isolation claims are false: the "restrict network" comment sets `LD_PRELOAD=` (`sandbox.go:257-260`), which does nothing to network; there is no seccomp/namespace/cgroup. Comment even admits "In production, this would use a proper WASM compiler service" (`sandbox.go:118-120`).
  - The pipeline runs **every fenced code block in skill markdown content** regardless of language (`internal/validation/pipeline.go:336-374`, `extractCodeBlocks:383-420`) — documentation snippets get executed.
  - **Reachability (honest boundary):** currently *latent* because `internal/validation` has zero importers (see G03); it becomes live the moment P4 wires `Validate()` into the create/expand/MCP paths (which the plan intends). A skill body is fully attacker-controllable via `POST /skills` and MCP `skill_create`.
- **Why it matters:** "sandbox validation" that runs untrusted input on the host, unauthenticated (G01), is the worst-case security defect; the naming is itself an anti-bluff violation (R11).
- **DECISION:** Before wiring P4, replace the process-fallback with a real isolation boundary (rootless Podman/gVisor per Constitution §11.4.161, or a true WASM runtime with `wasmtime`), and **fail-closed**: if no isolated runtime is available, `SKIP` with reason — never silently execute on the host. Do **not** execute arbitrary documentation code blocks; only execute snippets explicitly tagged as runnable POCs. Delete the `LD_PRELOAD` "network restriction" line. **Alternatives rejected:** (a) keep process fallback "for dev" — one config typo = host RCE; (b) drop code execution entirely — loses the P4.T3 sandbox gate the zero-bluff pipeline needs.
- **Test coverage:** security (network-egress attempt from snippet is blocked; filesystem write outside sandbox blocked; `fork`/resource limits enforced), integration (no-runtime ⇒ SKIP not execute), fuzz (malicious snippet corpus), mutation (remove isolation flag → egress test fails), regression. **Challenges:** yes (escape-attempt bank). **HelixQA:** yes.

- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `2befa77`. Mechanism: host code-execution DELETED BY CONSTRUCTION (no `os/exec`/`executeProcess`/`executeGoSnippet`/`WASMSandbox`/`LD_PRELOAD` remain; `StaticValidator` parses in-process only) — **not** that sandbox/WASM/Podman isolation was built. This closes G02 via a different mechanism than this item's own DECISION text (a real isolation boundary); the effect is equivalent-or-stronger for the stated host-RCE risk since host execution no longer exists at all.

G03 — Flagship pipelines are dead code: `internal/validation` (jury) and `internal/autoexpand` (auto-growth) are never instantiated; worker handlers are stubs

- **Category:** gap
- **Severity:** critical — R2 (dynamic creation), R8/R11 (zero-bluff validation), and the founding "central registrar + fully-automatic auto-growth jury" are unbuilt in the running system.
- **Evidence (as originally audited, 2026-07-15; see the dated STATUS note below for what this changed):**
  - `grep -rln skill-system/internal/validation` → **no match**; `grep -rln skill-system/internal/autoexpand` → **no match**. Neither `NewPipeline` is ever called. *(The* `internal/autoexpand` *half of this claim is superseded — see STATUS below; the* `internal/validation` *half was not touched by this fix round and is not re-audited here.)*
  - Worker "handlers" are explicit stubs: `internal/worker/runner.go:317` (`// Job handlers (stub implementations…)`), `handleAutoExpand` returns `Success:true` with no work (`runner.go:320-335`), `handleValidate` likewise (`runner.go:337-349`), `handleCodeAnalysis` likewise (`runner.go:351-363`). *(*`handleAutoExpand` *is superseded — see STATUS below;* `handleValidate`*/*`handleCodeAnalysis` *remain accurate in substance, current-tree line numbers* `runner.go:589-601`*/*`runner.go:603-615`*, banner comment now at* `runner.go:543`*.)*
  - The worker *cycles* only log: `runAutoExpandCycle` iterates and increments a counter but creates nothing (`runner.go:440-481`); `runValidationCycle` only logs (`runner.go:483-507`). *(Still accurate; current-tree line numbers* `runner.go:692-733` / `runner.go:735-759` *respectively — both shifted by the same fix round that landed* `handleAutoExpand`*, see STATUS below.)*
  - The stub comments assert work that never happens ("Actual expansion is done by the autoExpandWorker polling loop", `runner.go:332`; "Actual validation is done by the validationWorker", `runner.go:347`) — bluff-comments (R11).

(The "Actual expansion…" bluff-comment for `handleAutoExpand` *is REMOVED — see STATUS below. The "Actual validation is done by the validationWorker" bluff-comment for* `handleValidate` *remains, current-tree line* `runner.go:599` *— still false:* `runValidationCycle` *only logs, it does not validate.)*

- **Why it matters:** every skill created (REST or MCP) is written straight to the DB as `draft` with **no** resource-verify / sandbox / jury / cross-ref. The "zero-bluff guarantee" (`internal/validation/pipeline.go:1-4`) is not in force anywhere.
- **DECISION:** Wire `validation.Pipeline.Validate` and `autoexpand.Pipeline.Run` into the worker's real `handleValidate`/`handleAutoExpand` and into the create path; delete the stub comments; the worker cycles must call the pipelines, not log. Gate a "no skill reaches `validated`/`active` without a recorded jury verdict" invariant. **Alternatives rejected:** leaving pipelines as libraries "to be wired later" is the §11.4.197 un-wired-research failure the Constitution forbids.
- **Test coverage:** unit (pipeline stage state machine), integration (draft → jury → merge on real DB), e2e (create → validated only after ≥2 approvals), mutation (a fabricated skill must be rejected; strip a stage → test fails), regression. **Challenges:** yes (fabricated-skill-must-fail). **HelixQA:** yes (autonomous QA session over the pipeline).
- **STATUS (2026-07-17):** FULLY LANDED — all three G03 dispatch paths are now wired end-to-end:
  1. **Job-queue path**: `handleAutoExpand` (`runner.go`) dispatches through `r.autoexpand.Run(...)`; `handleValidate` dispatches through `r.validator.Validate(...)`; `handleCodeAnalysis` dispatches through `r.codeAnalyzer.AnalyzeProject(...)`.
  2. **Auto-expand ticker cycle**: `runAutoExpandCycle` now dispatches through `r.autoexpand.Run()` for each gap found (was log-only).
  3. **Validation ticker cycle**: `runValidationCycle` now dispatches through `r.validator.Validate()` AND promotes validated skills to active via the new `store.UpdateStatus` method (was log-only with a TODO for promotion).
     - `store.UpdateStatus` method added — transactional status change with audit log, used by validation worker to promote skills draft → active after passing all stages.
     - Worker embedder seam wired (prior round) — `NewRunner` calls `db.NewEmbedderFromConfig(cfg.Embedding)` and passes to both `store.WithEmbedder(emb)` and `autoexpand.NewPipeline(store, aeEmbedder, ...)`.
     - `DraftSkill` drafts via provider-agnostic `generateSkillDraft` (prior round).

- **Remaining G03 items**: `autoexpand.Pipeline.Run`'s own top-level gap-detection is structurally inert against any graph the store API constructs (tracked as G137); `internal/validation` package's own internal gaps tracked separately.

G04 — Zero automated tests exist; the R1 security fixes and every behaviour have no proof

- **Category:** test-coverage
- **Severity:** critical — direct violation of R8 (~100% across 13 test types + Challenges + HelixQA) and R11 (positive evidence only).
- **Evidence:** `find project -name '*_test.go'` → **0 files**. The CORS allowlist (`middleware.go:328-387`) and api_key redaction/removal (`middleware.go:230-250`, `287-292`) compile but have **no** behavioural test. `IMPLEMENTATION_PLAN.md:117` (P0.T2) *promises* such a test; none exists. `REQUIREMENTS.md:113` records "0 tests".
- **Why it matters:** every "green" claim is unbacked; regressions are undetectable; the anti-bluff covenant is unmet at the test layer.
- **DECISION:** Land P0.T3 test bootstrap first, then per-package tables with paired mutations (§1.1) as each package is wired, starting with the security middleware (behavioural proof: disallowed origin rejected, credentials never with `*`, api_key never logged) and the DAG/graph correctness. Coverage gate in CI. **Alternatives rejected:** deferring tests to a final P11 phase — the plan already forbids this (`IMPLEMENTATION_PLAN.md:328`); untested code has been shipping bugs G06/G07/G11.
- **Test coverage:** all 13 types are the deliverable here; prioritise unit + integration + contract + security + mutation first. **Challenges:** yes. **HelixQA:** yes.

HIGH

G05 — `LLMJury` auto-approves when no jury is configured (default state)

- **Category:** danger-zone / security-of-correctness
- **Severity:** high — the zero-bluff gate defaults to "approve everything".
- **Evidence:** `internal/validation/pipeline.go:428-439` — `if len(p.jury) == 0 { … Consensus: true … "no jury configured,`

`auto-approved" }`. No `ModelProvider` chain exists yet (P3 unbuilt), so the jury slice is empty by default.
- **Why it matters:** contradicts the founding "LLM jury (≥2 approvals)" and R11. A fabricated skill would pass the jury stage unconditionally.
- **DECISION:** Fail-closed — an empty jury when `validation.enabled` is a hard error (or forces `require_human_review`), never an auto-pass. Require `approval_threshold ≥ 2` real votes. **Alternatives rejected:** "auto-pass in dev" — same bluff class as G02's process fallback.
- **Test coverage:** unit (empty jury ⇒ fail/blocked, not pass), integration (2-of-3 real approvals required), mutation (flip auto-pass back → test fails). **Challenges:** yes. **HelixQA:** yes.
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `2befa77`. LLM jury now fails CLOSED (empty jury BLOCKS, never auto-approves); two-factor consensus, both factors tested.

G06 — `GetDependencyTree` returns only depth-1 children (recursive tree truncated)

- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `186e047`. Recursive cycle-guarded `GetDependencyTree` + MCP wire-in landed (supersedes the 2026-07-15 DESIGN DONE/Go-impl-PENDING note; design doc `research/g06_g07_skilltree_dag_design.md` preserved for history).
- **Category:** existing-bug
- **Severity:** high — the core "recursive dependency DAG" feature is broken; REST `/skills/:id/tree` and MCP `skill_tree` both under-report.
- **Evidence:** `internal/skill/graph.go:280-307` builds `childrenMap` for all depths but attaches **only** `root.Children = childrenMap[rootSkill.ID]` (`graph.go:306`); grandchildren `Children` are never populated. MCP's recursive serializer (`internal/mcp/tools.go:226-246`) therefore also emits a 1-level tree despite recursing. Contrast `GetAllDependencies` (`graph.go:347-371`) which is correct (flat closure).
- **Why it matters:** the founding requirement is "endless, deeply-recursive skill branching"; the tree API silently returns a shallow slice — wrong results presented as complete.
- **DECISION:** Assemble the full tree from `childrenMap` recursively (attach children to every node by ID, cycle-guarded), or select `parent skill_id` in the CTE and build the tree in one pass. Add depth + cycle tests on the seed corpus. **Alternatives rejected:** documenting it as "direct deps only" — contradicts the API contract (`api/openapi.yaml:1355-1370` `SkillTreeNode` is recursive).

- **Test coverage:** unit (tree assembly), integration (seed android closure returns known N-level tree), property (tree node count == closure size), regression, mutation (revert to depth-1 → test fails). **Challenges:** yes.

G07 — TOML/JSON dependency+resource round-trip is broken (edges silently dropped on import)

- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `073192f`. Full 6-type TOML dependency/resource round-trip landed, no-silent-loss + strict-decode (supersedes the 2026-07-15 DESIGN DONE/Go-impl-PENDING note; design doc `research/g06_g07_skilltree_dag_design.md` preserved for history).
- **Category:** existing-bug
- **Severity:** high — breaks R14 git-versionable round-trip and the R6 wizard's DAG mapping.
- **Evidence:**
  - REST `POST /skills` discards `req.Deps` and `req.Resources` entirely — the built `models.Skill` sets only scalar fields (`internal/api/skills_handler.go:165-176`); `CreateDepsRequest`/`CreateResourceReq` (`skills_handler.go:33-45`) are parsed then ignored.
  - REST import: `convertTOMLWrapper` creates `SkillDependency{RelationType:…}` but **never sets `DependsOnName`/`DependsOn`** — the target name is thrown away (`skills_handler.go:548-565`, literally `_ = depName // placeholder for resolution`).
  - Export *does* write `DependsOnName` (`skills_handler.go:611-616`), so export→import is non-idempotent: names go out, come back empty.
  - **UNCONFIRMED:** the MCP create path uses `skillStore.ImportFromTOML` (`internal/mcp/tools.go:275`), a different function in `internal/skill/import_export.go` not read in this pass — needs verification that it preserves edge names.
- **Why it matters:** a skill graph whose *edges* vanish on ingest cannot resolve dependencies, run the recursive CTE, or drive the wizard; the DB drifts from the git-versioned TOML source of truth.
- **DECISION:** Resolve dependency names → IDs at create/import (insert edges via `Store.AddDependency`, which already validates + cycle-checks, `graph.go:21-96`); persist resources; make export→import a proven identity round-trip. **Alternatives rejected:** deferring edge resolution to a later "linker" pass — leaves the graph disconnected in the interim and hides the drift.

- **Test coverage:** unit (convertTOMLWrapper preserves names), integration (create with deps → edges in DB), property/round-trip (export→import→export byte-stable), contract, mutation (drop `DependsOnName` → round-trip test fails). **Challenges:** yes. **HelixQA:** yes.

G08 — No TOON codec exists; wire format mandate unmet; OpenAPI still on JSON+TOML

- **Category:** gap / spec-drift
- **Severity:** high — R/founding mandate "TOON primary + JSON fallback"; danger-zone #2 (silent fallback).
- **Evidence:** `grep -rin toon --include='*.go'` → **0 matches** in the whole backend. Content negotiation only knows JSON/TOML (`internal/api/middleware.go:114-148`, `response.go`). `api/openapi.yaml` still advertises `application/json` + `application/toml` throughout (e.g. `openapi.yaml:21-29`, `111-113`, `1043-1051`), never `application/toon`. `IMPLEMENTATION_PLAN.md:335` flags the exact risk; P1.T2/P7.T4 are queued not done.
- **Why it matters:** clients told to expect TOON will get TOML/JSON with no signal — the silent-fallback bluff the plan warns about.
- **DECISION:** Implement/vendor a spec-conformant Go TOON codec (`github.com/toon-format/toon`) with its own golden test vectors before advertising `application/toon`; until it exists, the API MUST NOT claim TOON. Revise `openapi.yaml` content-negotiation in the same change (P7.T4). **Alternatives rejected:** "interpret TOON as TOML" — explicitly superseded (`REQUIREMENTS.md:38-40`).
- **Test coverage:** unit (round-trip struct→TOON→struct), contract (golden TOON fixtures byte-for-byte), integration (`Accept: application/toon` returns TOON; unknown → JSON fallback with correct `Content-Type`), fuzz (malformed TOON rejected), mutation (swap codec to JSON → golden test fails). **Challenges:** yes.
- **STATUS (2026-07-16):** `Implemented (→ Fixed.md)` — `2785e11`. TOON codec + `application/toon` content negotiation landed.

G09 — Pervasive OpenAPI ↔ implementation drift; most documented endpoints are unimplemented or differently shaped

- **Category:** spec-drift
- **Severity:** high — the contract is the "single source of truth" for R3 thin clients (`REQUIREMENTS.md:156`); clients generated from it will not work.

- **Evidence (spec → live route in `cmd/server/main.go:169-243`):**
  - `POST /search` (`openapi.yaml:506-516`) → live is `GET /api/v1/skills/search?q=` (`main.go:182`). Also the hardened `internal/api/server.go:182` registers `GET /search` — both disagree with the POST spec.
  - `GET /registry/missing` (`openapi.yaml:610`) → hardened route is `GET /registry/missing-deps/:id` (`internal/api/server.go:189`); live server has `GET /api/v1/missing` (`main.go:234`).
  - `POST /expand/{name}` (`openapi.yaml:703`) → hardened `POST /expand` no name param (`server.go:198`); live server has **no** expand route at all.
  - `POST /learn` + `GET /evidences/{skill_name}` (`openapi.yaml:807, 874`) → hardened `POST /learn/projects`, `GET /learn/evidences/:skill_id` (`server.go:206-208`); live server has neither.
  - `/health`, `/metrics`, `/version` are documented under `/api/v1` with `ApiKeyAuth` + 401 (`openapi.yaml:36, 913-972`) but live are top-level and unauthenticated (`main.go:151`, `server.go:155-157`).
  - Live server implements only list/search/get/tree/coverage/missing (`main.go:170-243`) — no create/update/delete/import/export via REST.
- **Why it matters:** contract-first codegen (P8) will produce clients calling endpoints that 404 or expect the wrong verb/shape.
- **DECISION:** Treat `openapi.yaml` as authoritative, regenerate handlers/routes to match, and add a contract-test gate (schema-validate every response, assert route table == spec) that fails CI on drift. Do this **after** G01 collapses to one server. **Alternatives rejected:** hand-syncing docs to code — drift returns immediately without a gate.
- **Test coverage:** contract (spec-validation per endpoint + route-parity), integration, regression, smoke, mutation (rename a route → parity test fails). **Challenges:** yes. **HelixQA:** yes.

G10 — Embedding dimension: no model↔column assertion; `vector(768)` hard-coded; OpenAI vector length unchecked; non-openai/local providers unsupported

- **STATUS (2026-07-15):** DESIGN DONE + spot-verified vs `255061b` (`vector(768)` hard-coded at `001_initial.up.sql:14`) → `research/g10_embedding_provider_design.md` (23 tests; also covers G27 `EmbedAsync`/`sanitizeTableName`). Decision = `(provider,model)→dim` registry + fail-closed boot-time `AssertEmbeddingDimension` (§11.4.201) + templated `vector(N)` + OpenAI length check + `validateTableName` reject-not-strip. Go impl PENDING — needs

embedder-construction wiring (`cmd/*`); composes P1.T1 additively (zero embedding-DDL overlap).

- **Category:** danger-zone / inconsistency
- **Severity:** high — the 768/1536/384 conflict is "resolved" only by two unenforced constants that can silently disagree at runtime.
- **Evidence:**
  - SQL columns hard-code `vector(768)` (`migrations/001_initial.up.sql:14, 60`); config default `Dimensions: 768` (`internal/config/config.go:174`). They agree **by coincidence**, not by construction — the migration is not templated from config.
  - No startup assertion that `embedder.Dimensions() == column dim` exists (danger-zone #3, `IMPLEMENTATION_PLAN.md:336`, is unmet). `config.validate` only checks `dimensions > 0` (`config.go:407-409`).
  - `OpenAIEmbedder.Embed` never verifies the returned vector length equals `e.dimensions` (`internal/db/embedding.go:124-143`) — contrast `LocalEmbedder` which does (`embedding.go:257-262`). A model returning 1536 will be handed to a `vector(768)` insert → runtime failure.
  - `NewEmbedderFromConfig` supports only `"openai"`/`"local"` (`embedding.go:294-308`), but `SPEC.md:396` config lists `anthropic` and R7 mandates HelixLLM/LLMsVerifier providers — those configs error at startup.
- **Why it matters:** a single config edit (e.g. switching to a 1536-native model) produces opaque insert failures; the blueprint's 1536 assumption is silently incompatible.
- **DECISION:** Pin **768** as the shipped default (matches SPEC + current column + research "sweet spot", `SPEC.md:12`); make `vector(N)` a migration parameter driven by `config.embedding.dimensions`; add a **startup assertion** that queries `information_schema`/`pg_attribute` for the column's declared dim and fails fast on mismatch; add an OpenAI response-length check; extend the provider factory for HelixLLM/OpenAI-compatible providers (R7). **Alternatives rejected:** (a) support all dims live via runtime re-index — expensive, out of MVP scope; (b) leave 768 hard-coded and undocumented — reproduces the drift.
- **Test coverage:** unit (provider factory incl. helixllm; OpenAI length mismatch rejected), integration (startup fails on dim mismatch; correct dim inserts), contract (config schema), mutation (change column to 1536, keep config 768 → startup assertion fails). **Challenges:** yes.

G11 — Worker does no real work and can panic the process (unchecked type assertions in a recover-less goroutine)

- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `0e255b4`. Worker panic firewall landed (recover + backoff-restart wrapper on every worker goroutine + per-job firewall); supersedes the 2026-07-15 DESIGN DONE/Go-impl-PENDING note (design doc `research/g11_worker_design.md` preserved for history).
- **Category:** existing-bug / gap
- **Severity:** high — background auto-growth/validation/review are non-functional; a crash vector exists.
- **Evidence:** stubs at `internal/worker/runner.go:317-368`; cycles that only log (`runner.go:440-507`). `runRegistryReview` does `coverage["total_skills"].(int)` and `coverage["coverage_percentage"].(string)` (`runner.go:518-519`) — unchecked assertions; if `GetCoverage` returns a differently-typed/absent key the goroutine panics, and worker goroutines have **no `recover()`** (`runner.go:375-434`), so the process dies. (The API `Recovery()` middleware, `middleware.go:254-276`, does not cover worker goroutines.)
- **Why it matters:** the "central registrar + fully-automatic auto-growth" is the product's core promise (R2) and it is inert; the panic risk turns a data-shape change into an outage.
- **DECISION:** Implement the handlers/cycles against `autoexpand/validation` (see G03); replace unchecked assertions with comma-ok + typed struct returns from `GetCoverage`; add a `recover()` + restart wrapper around every worker goroutine. **Alternatives rejected:** keeping stubs "until P4/P5" — un-wired-research (§11.4.197) and the panic ships regardless.
- **Test coverage:** unit (cycle calls pipeline; coverage type-safety), integration (worker creates a real skill from a seeded gap), chaos (malformed coverage map ⇒ logged error not panic), mutation (reintroduce bare assertion → panic test fails). **Challenges:** yes. **HelixQA:** yes.

G12 — tree-sitter is a stub: native parsing always fails; regex-only; Kotlin/C# unsupported despite being configured

- **STATUS (2026-07-17):** COMPLETED — CGO/native tree-sitter split landed (`4f5fdd5`). `treesitter_native.go` (584 lines, `//go:build cgo`) implements real native parsing via `github.com/tree-sitter/go-`

`tree-sitter` with grammars for Go, Python, Java, JavaScript, C, C++, Rust, C#, and Kotlin. `treesitter_native_stub.go` (`//go:build !cgo`) preserves the regex-only fallback. `cgoAvailable` package-level var replaces the old `cgoEnabled()` stub. `initNativeParser` now constructs real `sitter.Parser` + sets language via grammar module's `Language()`. `parseNative` builds real AST (`Tree.Root` with `TSNode` tree). `extractImportsNative`/`extractFunctionsNative`/`extractClassesNative` walk the real AST via tree-sitter queries. `FidelityNative` set on all native-parse results. All 13 tests pass (including #6 CGO native path — now exercises real tree-sitter). Full suite GREEN (24 packages, CGO_ENABLED=1). Bash/Dart same-class defect tracked as follow-up per design doc §5.
- **Category:** gap
- **Severity:** high — R2 requires tree-sitter as a *working POC, not a stub*; learn-from-codebase (R2/R6/P5) rests on it.
- **Evidence:** `initNativeParser` **always** returns an error (`internal/codeanalysis/treesitter.go:106-131`); `parseNative`/`extractImportsNative`/`extractFunctionsNative`/`extractClassesNative` all return `"not implemented"` (`treesitter.go:160, 230, 235, 240`). Only regex fallback runs. `compilePatterns` has **no `kotlin` or `csharp` case** (`treesitter.go:264-296`), yet `kotlin` is in the default analysis languages (`config.go:194`) and normalizeLanguage maps `kt` → `kotlin` (`treesitter.go:558-559`) — Kotlin files yield an empty pattern set ⇒ zero extraction.
- **Why it matters:** "learn from real codebases" over Java/Kotlin/C++ (R13 corpus) will silently extract little/nothing for Kotlin and rely on brittle regex for the rest — evidence quality the jury cannot trust.
- **DECISION:** Land a CGO tree-sitter build (grammars for the R13 languages) behind a build tag, with the regex parser as an explicit, labelled fallback that reports reduced fidelity (never silently); add Kotlin/C# patterns to the fallback in the interim. Prove extraction on a real repo (P5.T2 gate). **Alternatives rejected:** shipping regex-only as "tree-sitter" — an anti-bluff/naming violation (R11).
- **Test coverage:** unit (per-language extraction incl. kotlin), integration (parse a real Android/Kotlin repo → real symbols), fuzz (malformed source doesn't crash), mutation (remove a grammar → extraction test fails). **Challenges:** yes.

G13 — Two rival `docker-compose.yml` files (rival-copy risk)

- **STATUS (2026-07-16):** `Completed (→ Fixed.md)` — `9b85df2`. Compose canonicalization onto `deploy/` with app+monitoring profiles landed, closing G13's own scope (the sibling G17/G22/G23/G24 decisions

recorded in this same design pass are tracked under their own ids, see below); design doc `research/ops_hardening_design.md` preserved for history.
- **Category:** ops
- **Severity:** high — P12.T4 explicitly requires one canonical compose; two exist now.
- **Evidence:** `project/docker-compose.yml` (9198 bytes) **and** `project/deploy/docker-compose.yml` (3332 bytes) both present. `IMPLEMENTATION_PLAN.md:258` (P12.T4) calls for exactly one; the scripts/systemd unit must reference a single file.
- **Why it matters:** divergent compose files drift (ports, image tags, env, volumes); operators/scripts pin the wrong one; reproducibility breaks.
- **DECISION:** Choose `project/deploy/` as the canonical ops home (it already carries `.env.example`, `systemd/`), delete/merge the root `docker-compose.yml`, and have `scripts/*` + the `systemd --user` unit reference only the canonical path. **Alternatives rejected:** keeping both "for dev vs deploy" — the exact rival-copy anti-pattern; use compose overrides/profiles instead if two modes are truly needed.
- **Test coverage:** smoke (compose up → `pg_isready`, `CREATE EXTENSION vector`, `SELECT 1`), integration (scripts reference the one file), regression (grep gate: no second compose), acceptance (systemctl –user up/down cycle). **Challenges:** yes.

G14 — Submodule policy conflict (§11.4.28C single-canonical vs operator parent-priority+both-synced) unresolved; all 7 declared deps unvendored

- **Category:** supply-chain / ops
- **Severity:** high — an open governance escalation blocks P3/P7/P8/P10/P11/P13 dependency work.
- **Evidence:** `project/helix-deps.yaml` header asserts §11.4.28C "ONE canonical location … Nested own-org submodule chains are FORBIDDEN"; `REQUIREMENTS.md:76-79` (R9) wants "parent-dir versions have PRIORITY. BOTH copies must always be in sync"; `IMPLEMENTATION_PLAN.md:286` (X1) records the escalation as **open** ("no second-copy `--apply` until resolved"). The manifest declares 7 deps (llms_verifier, helix_llm, helix_agent, embeddings, helix_qa, challenges, docs_chain) all `layout: grouped` — none are vendored yet (`submodules/` absent).
- **Why it matters:** the two policies are literally contradictory (one canonical copy vs two synced copies); acting on either without a decision risks a §9.2 data-safety/duplication mistake and blocks R7/R8/R10.

- **DECISION (recommended reconciliation):** treat the **parent-ecosystem copy as the single logical canonical** and any `submodules/<name>/` as a **read-only mechanical mirror** pinned to the same commit by `sync_submodules.sh` (verify-only by default, `--apply` gated). This satisfies §11.4.28C's "one canonical" *semantically* (one source of truth) while honouring the operator's "parent priority + both in sync" *operationally* (the mirror is derived, never independently edited). Escalate this exact framing to the operator for sign-off before any `--apply`. **Alternatives rejected:** (a) two independently-editable copies — violates §11.4.28C and invites divergence; (b) delete the parent copy and vendor only under `submodules/` — violates the operator's parent-priority mandate.
- **Test coverage:** integration (`sync_submodules.sh` dry-run resolves each dep to exactly one canonical + a pinned mirror), security (fail-closed on unexpected path — already hardened, `REQUIREMENTS.md:197`), regression (`ls-remote` reachability per dep), mutation (introduce a nested own-org submodule → sync fails). **Challenges:** yes.

G15 — Aurora OS / HarmonyOS client feasibility unproven (highest client risk)

- **Category:** danger-zone / gap
- **Severity:** high — R3 hard-requires Aurora + HarmonyOS clients; the plan itself ranks this the top danger.
- **Evidence:** `IMPLEMENTATION_PLAN.md:334` (danger-zone #1) and P8.T5 (`:215`) both flag Aurora/Harmony as "smallest ecosystem, highest risk"; `REQUIREMENTS.md:160` build order ends "Aurora last (highest risk)". The only proposed path is Flutter-OHOS + the omprussia embedder (`REQUIREMENTS.md:158-160`) — a spike, not a proven build. No client code exists (P8 not started).
- **Why it matters:** if the embedder path is infeasible, R3 cannot be met on those OSes and must be re-scoped honestly, not bluffed.
- **DECISION:** Run the Flutter-OHOS + omprussia spike **early** (before committing the shared-core client architecture), and risk-flag with the **exact blocker** (no bluffed build) if it fails, per §11.4.112 (structural-impossibility gets an evidence-backed won't-fix) rather than a silent gap. Because every surface is a thin OpenAPI/MCP client, the backend contract is the de-risking asset — freeze it first. **Alternatives rejected:** building Aurora last "and hoping" — the plan already warns against; deferring the spike hides the risk until the end.
- **Test coverage:** acceptance (one build artifact per OS or a documented blocker), smoke (thin client hits `/health` + wizard),

contract (generated client compiles against OpenAPI). **Challenges:** yes (per-OS build feasibility). **HelixQA:** device-lab dependent — flag as operator-attended where autonomous build is infeasible.

MEDIUM

G16 — `sandbox_type = "wasm"` never actually uses WASM; Docker sandbox has conflicting mounts

- **Category:** weakness
- **Severity:** medium
- **Evidence:** even with a runtime present, the Go path is `go run` (`internal/validation/sandbox.go:132-136`) and non-Go langs fall to `executeProcess` (`sandbox.go:138-139`); "WASM" is a misnomer. `DockerSandbox.Execute` mounts `-v tmpDir:/tmp:ro` **and** `--tmpfs /tmp:…` (`sandbox.go:407-408`) — the tmpfs shadows the read-only bind, so `go run /tmp/main.go` reads an empty `/tmp` (`sandbox.go:387`).
- **Why it matters:** the isolation story is inconsistent and the Docker path is subtly broken for the Go case.
- **DECISION:** Rename/replace per G02; if Docker is the isolation boundary, mount code at a distinct path (e.g. `/work:ro`) and drop the conflicting tmpfs, or pass code via stdin. **Alternatives rejected:** leaving the dual `/tmp` mount — non-deterministic behaviour.
- **Test coverage:** integration (Docker path runs the mounted file), unit (mount args well-formed), mutation (reintroduce dual /tmp → test fails).
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `2befa77`. Same commit as G02 — host code-execution DELETED BY CONSTRUCTION (removes the Go `go run`/dual-`/tmp`-mount problem by deleting the executable path entirely, not by fixing the mount).

G17 — Weak/committed default DB password; config validation misses provider/sandbox enums

- **Category:** weakness / security
- **Severity:** medium
- **Evidence:** `internal/config/config.go:167` defaults `Password: "secret"`; `config/config.toml` ships `password = "secret"`. `config.validate` (`config.go:390-433`) never validates `embedding.provider`, `validation.sandbox_type`, `logging.level`, or

`mcp.transport` against their allowed sets — a typo (`provder = "opennai"`) fails late, deep in the call stack.
- **Why it matters:** weak default invites deployment as-is; unvalidated enums produce confusing runtime errors.
- **DECISION:** Require the DB password via env with **no** working default (fail-closed if unset in non-dev); validate all closed-set config fields in `validate()`. `deploy/.env` is correctly git-ignored (`.gitignore:21`) and untracked — keep it so. **Alternatives rejected:** documented default password — a standing credential-hygiene risk (§11.4.10).
- **Test coverage:** unit (invalid provider/sandbox/level rejected; empty password rejected in prod mode), security (no secret in tracked files — pre-commit grep), mutation (add an invalid enum → validate fails).
- **STATUS (2026-07-15):** DESIGN DONE → `research/ops_hardening_design.md`; impl PENDING.

G18 — CORS allowlist unreachable on the live path; SPEC config sample omits `allowed_origins`

- **Category:** weakness / spec-drift
- **Severity:** medium
- **Evidence:** the hardened, config-driven allowlist lives in `internal/api` (`middleware.go:328-387`, fed by `ServerConfig.AllowedOrigins`, `config.go:59-63`) but the live server uses `corsMiddleware()` wildcard (`cmd/server/main.go:362-373`) and never reads `AllowedOrigins`. `SPEC.md:376-384` config sample has no `allowed_origins`, and `config/config.toml` — the shipped template — likewise omits it, so operators won't know to set it.
- **Why it matters:** once G01 is fixed, a browser client still breaks unless `allowed_origins` is documented and set; today it's wildcard-open.
- **DECISION:** Wire `AllowedOrigins` end-to-end (config→ServerConfig→CORS), document it in `config.toml` + SPEC §8 with a safe example, default empty (fail-closed). **Alternatives rejected:** wildcard default — the security posture R1 removed.
- **Test coverage:** integration (config allowlist honoured), security (non-allowlisted origin blocked), contract (config documents the key).
- **STATUS (2026-07-15):** DESIGN DONE + verified vs `255061b` → `research/g18_g25_g26_correctness_bundle.md`. Security half CLOSED by G01 (`cmd/server/main.go:151` `router.Use(api.CORS(cfg.Server.AllowedOrigins))`; wildcard `corsMiddleware` deleted; empty-allowlist proven live by `cmd/server/`

`security_test.go:105 TestNoWildcardCORSOnLivePaths`). Two residuals OPEN: (a) `SPEC.md` §8 config sample still omits `allowed_origins`/`api_keys`/`auth_disabled` (G01 touched no `.md`); (b) no live-`buildRouter` test for a POPULATED allowlist. Narrowed to the SPEC doc update + 1 integration test; runtime-wiring clause closable per §11.4.90 (`superseded-by-later-mandate`, cites G01 `1a1a3f3`). Impl PENDING.

G19 — `SPEC.md §8` config sample uses `--` comments (invalid TOML)

- **Category:** spec-drift / doc
- **Severity:** medium (doc only; the real file is correct)
- **Evidence:** `SPEC.md:381, 396, 404, 407, 425` use `-- comment` syntax inside a `config.toml` block; TOML comments are `#`. The actual `config/config.toml` correctly uses `#`. A reader copy-pasting the SPEC sample gets a file that fails `toml.DecodeFile` (`config.go:251`).
- **Why it matters:** nano-detail-docs mandate (founding) is undermined by a sample that won't parse; erodes trust in the spec.
- **DECISION:** Fix the SPEC §8 sample to `#` comments and add a docs lint that TOML-parses fenced `toml` blocks in the docs (composes with Docs Chain, R10). **Alternatives rejected:** leaving it — a latent copy-paste footgun.
- **Test coverage:** unit/lint (parse every ```toml block in docs), regression.
- **STATUS:** `Fixed (→ SPEC.md)` — 2026-07-17. All 8 `-- comment` usages in TOML blocks (SPEC.md §8 config sample) changed to `# comment`. The `--` comments in the SQL block (§5) are valid SQL syntax and were left unchanged.

G20 — Auto-expand fabricates placeholder skills without an LLM; couples to concrete `*OpenAILLM`; drafted resources never persisted

- **Category:** weakness / gap (latent — package unwired per G03)
- **Severity:** medium
- **Evidence:** with `p.llm == nil` (the default until P3), `DraftSkill` returns `createMinimalDraft` (`internal/autoexpand/pipeline.go:209-213`), which stores boilerplate content ("This skill was auto-generated to fill a gap", `pipeline.go:282-311`) as a real skill — fake knowledge (R11). `DraftSkill` type-asserts `p.llm.(*OpenAILLM)` and errors on any other `LLMClient` (`pipeline.go:215-218`), defeating the interface and R7 pluggability.

In `Run`, drafted `resources` get a `SkillID` assigned but are **never persisted** (`pipeline.go:401-403`).
- **Why it matters:** once wired, auto-growth would flood the graph with placeholder skills and drop their resources — the opposite of the zero-bluff promise.
- **DECISION:** Never persist a placeholder as a real skill — either produce genuine LLM content or mark the gap as unfilled; program to the `LLMClient` interface (remove the concrete assertion); persist resources in the same transaction as the skill. **Alternatives rejected:** keeping the minimal-draft fallback for "graceful degradation" — degrades into bluff data.
- **Test coverage:** unit (nil LLM ⇒ no placeholder persisted; interface pluggability), integration (draft → resources persisted), mutation (reintroduce placeholder-persist → anti-bluff test fails). **Challenges:** yes.
- **STATUS (2026-07-15):** DESIGN DONE + all file:line claims verified vs `255061b` → `research/g20_autoexpand_realgrowth_design.md`. Confirmed: `internal/autoexpand/pipeline.go:211/226` `createMinimalDraft`, `:215` `p.llm.(*OpenAILLM)` concrete assertion, `:282` the placeholder fabricator; `llm.go:26` `LLMClient` interface; `NewLLMClientFromConfig` factory CONFIRMED ABSENT everywhere (the R19 plug-in point). Decision = delete `createMinimalDraft`, program to `LLMClient` (no `*OpenAILLM` assertion), transactional `Store.CreateWithResources`, compose G05 jury. Composes R19 (`research/r19_anthropic_api_support_design.md` — Anthropic as an `LLMClient` + the missing factory). *(The* `pipeline.go:215-218` *concrete-assertion citation in this note is now STALE — see the dated STATUS note below, which supersedes the "Go impl PENDING" verdict for the assertion half specifically.)*
- **STATUS (2026-07-16):** PARTIALLY LANDED — the concrete-`*OpenAILLM`-assertion half of this item is FIXED this round (via G03 fix round): `DraftSkill` (`pipeline.go:232`) now drafts through the provider-agnostic `generateSkillDraft` (`internal/autoexpand/llm.go:249`, which the exported `OpenAILLM.GenerateSkillDraft`, `llm.go:216`, itself delegates to) instead of asserting `p.llm.(*OpenAILLM)` — the assertion no longer exists anywhere in `pipeline.go`, so the previous `pipeline.go:215-218` citation is stale; ANY configured `LLMClient` (including `*AnthropicLLM`, per R19/G28) now drafts successfully. **The type-assertion coupling is resolved — G03 worker-dispatch wiring was the enabling fix.** Still OPEN, unchanged by this round: the no-LLM `createMinimalDraft` placeholder-persist fallback (`pipeline.go:294-323`, called from the `p.llm == nil` branch at `pipeline.go:219` and from the LLM-error fallback at

`pipeline.go:238`) is untouched — a placeholder skill is still persisted as a real skill on either path; and drafted `resources` are still never persisted, only `SkillID`-stamped in memory (`pipeline.go:362-367`, comment explicitly notes "persistence of the resources themselves is a separate, pre-existing follow-up"). Composes G03 (this is the fix that let G03's worker-dispatch wiring land — a nil-`LLMClient`-assertion error on every non-OpenAI provider would otherwise have broken any `"anthropic"`-configured worker's draft path).

- **STATUS (2026-07-17):** COMPLETED — all three G20 defects CLOSED. (1) **Placeholder persist deleted**: `DraftSkill` now returns an error when `p.llm == nil` (instead of falling through to `createMinimalDraft`); the LLM-error branch likewise returns the error instead of silently degrading to a placeholder. No placeholder content is ever persisted as a real skill. (2) **Resources persisted**: `draftPersistAndCrossReference` calls `p.store.BulkAddResources` after creating the skill; resources are written to the DB, not just `SkillID`-stamped in memory. (3) **Type-assertion already removed in prior round**. Anti-bluff regression test `pipeline_crossreference_test.go` updated to assert error-on-nil-LLM (not placeholder persist); verified that `store.GetByName` returns nothing on the nil-LLM path. Cross-reference test (`TestDraftPersistAndCrossReference_PersistsAndCrossReferences_RequiresLiveDatab` confirms no phantom skill row is created. All `-short -race` tests GREEN across 23 packages.

G21 — Resource verification is shallow (HEAD-only, best-effort hash, fail-open on fetch errors)

- **Category:** weakness
- **Severity:** medium
- **Evidence:** `verifySingleResource` (`internal/validation/pipeline.go:259-303`) passes on any HEAD `< 400`; the content-hash check only runs when a prior hash exists and returns `nil` (pass) on any GET/read error (`pipeline.go:280-292`). SSRF is possible — arbitrary skill-supplied URLs are fetched server-side with no allowlist.
- **Why it matters:** "source verification" (stage 1 of the zero-bluff pipeline) can be satisfied by any reachable URL; a moved/altered doc without a stored hash passes; skill-controlled URLs enable SSRF.
- **DECISION:** Require a stored hash for `official-doc`/`code` resources, treat fetch/read errors as verification failures (not pass), and add an egress allowlist / block link-local + metadata IPs (SSRF guard).

Alternatives rejected: HEAD-only reachability as sufficient — it proves nothing about content (R11).
- **Test coverage:** unit (dead URL fails, mismatched hash fails, fetch error fails-closed), security (SSRF to 169.254.169.254 blocked), integration, mutation (flip fail-open back → test fails). **Challenges:** yes.
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `2befa77`. SSRF egress guard now blocks the full private/reserved space (RFC1918/ULA/link-local/metadata) + `additionalBlockedRanges`, re-screened on every dial via `net.Dialer.Control`.

G22 — No rate limiting / auth on the live server; body limit only; Brotli flush errors ignored

- **Category:** weakness / performance
- **Severity:** medium
- **Evidence:** P7.T5 (auth + rate limiting, `IMPLEMENTATION_PLAN.md:203`) is unbuilt; the live server has neither (`cmd/server/main.go:140-283`). `MaxBodySize(100MB)` is applied only in the hardened (dead) path (`internal/api/server.go:149`), not live. Brotli `Flush()`/`Close()` return values are discarded (`internal/api/middleware.go:106-107`), so a compression error yields a silently truncated response.
- **Why it matters:** unauthenticated + unthrottled + code-executing endpoints (post-G03 wiring) are a DoS/abuse surface; silent truncation corrupts responses.
- **DECISION:** Add token-bucket rate limiting + the 100MB body cap to the unified server (G01); handle Brotli errors (abort the response on failure). **Alternatives rejected:** relying on an upstream proxy for limits — the app must be safe standalone per the deploy model (systemctl –user, R15).
- **Test coverage:** load (429 over-limit), integration (413 over-size), unit (Brotli error handled), security, regression.
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `e81a493`. Rate-limit + body-cap + Brotli-error hardening landed.

G23 — Migrations loaded from a cwd-relative path; failure only warns and the server continues

- **Category:** ops
- **Severity:** medium
- **Evidence:** `db.Migrate(ctx, pool, "./migrations")` (`cmd/server/main.go:84`) is cwd-relative; on failure the server logs `Warn` and

keeps running (`main.go:85-88`), so it can serve traffic against a schema-less DB and fail every query.
- **Why it matters:** silent boot on a broken schema is a §11.4.108 runtime hazard; running from a different directory skips migrations entirely.
- **DECISION:** Resolve the migrations dir from config/embed (`embed.FS`), and **fail-fast** (exit non-zero) if migrations don't apply. **Alternatives rejected:** warn-and-continue — hides a fatal state.
- **Test coverage:** integration (missing migrations dir ⇒ startup fails), smoke (`migrate up` on fresh pgvector DB, `\d+` verified), regression.
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `ffada37`. `embed.FS` migrations + fail-closed startup landed.

LOW

G24 — Health/metrics/version unauthenticated; `/metrics` exposes Prometheus internals publicly

- **Category:** security
- **Severity:** low
- **Evidence:** `cmd/server/main.go:151` (`/health`), `internal/api/server.go:155-157` register these outside auth; OpenAPI marks them `ApiKeyAuth`/401 (`openapi.yaml:913-972`). `/metrics` returns full Prometheus exposition (`middleware.go:22-52` counters).
- **Impact:** minor info-leak (internal metrics, versions) to anonymous callers.
- **DECISION:** Keep `/health` open (liveness), but gate `/metrics` behind auth or bind it to a private interface; align `/version` with the contract. **Alternatives rejected:** authing `/health` — breaks orchestrator probes.
- **Test coverage:** security (anonymous `/metrics` denied where required), contract, regression.
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `7e70754`. `/health`/`/metrics`/`/version` hardening + OpenAPI reconciliation landed (current HEAD).

G25 — `RemoveDependency` ignores name-lookup errors → audit log with empty names

- **Category:** weakness
- **Severity:** low

- **Evidence:** `internal/skill/graph.go:103-104` discard the `Scan` error via `_ =`; if a skill is already gone, the audit entry records empty `from`/`to`.
- **Impact:** degraded audit fidelity (R11 evidence trail).
- **DECISION:** Capture names best-effort but record the not-found condition explicitly in the audit detail. **Alternatives rejected:** ignoring silently — weakens the audit trail.
- **Test coverage:** unit (audit detail records missing name), regression.
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `67ce4d6`. `buildRemovalAuditDetail` helper landed — audit-log empty-name-on-lookup-error fixed; supersedes the 2026-07-15 DESIGN DONE/Go-impl-PENDING note (design doc `research/g18_g25_g26_correctness_bundle.md` preserved for history).

G26 — `${VAR:-default}` cannot resolve to an intentionally-empty value; provider/model env-substitution edge cases

- **Category:** weakness
- **Severity:** low
- **Evidence:** `interpolate` treats any unset-or-empty env as "use default" (`internal/config/config.go:342-348`) — an env var explicitly set to `""` falls through to the default, so an operator cannot blank a value via env.
- **Impact:** surprising config behaviour for empty overrides.
- **DECISION:** Distinguish "unset" (`os.LookupEnv`) from "empty" so an explicit empty override is honoured. **Alternatives rejected:** documenting the quirk — still astonishing.
- **Test coverage:** unit (empty-override honoured; unset uses default), regression.
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `fb94352`. `os.Getenv` → `os.LookupEnv` switch landed — explicit-empty override now honored; supersedes the 2026-07-15 DESIGN DONE note (design doc `research/g18_g25_g26_correctness_bundle.md` preserved for history).

G27 — `sanitizeTableName` silently strips instead of rejecting; `EmbedAsync` result-channel semantics

- **Category:** weakness
- **Severity:** low
- **Evidence:** `internal/db/vector.go:288-296` strips non-alnum chars rather than rejecting (`"skills; DROP"` → `"skillsDROP"`); safe today

because callers pass internal constants only (`vector.go:216, 226`), but a future dynamic caller could hit a wrong-but-valid table name. `EmbedAsync` (`internal/db/embedding.go:359-405`) is correct (buffered to `len(texts)`), noted only as a caller-contract reminder.
- **Impact:** latent foot-gun if table names ever become user-influenced.
- **DECISION:** Reject invalid table names outright (return error) and keep the caller set to a fixed allowlist enum. **Alternatives rejected:** silent stripping — masks programmer error.
- **Test coverage:** unit (invalid table name rejected), security, regression.
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `e48b5a4`. `sanitizeTableName` replaced with `validateTableName` (reject, not strip).

G28 — Anthropic Messages API as a first-class `LLMClient` provider (R19); `NewLLMClientFromConfig` factory absent

- **Category:** feature / gap (R19 — operator mandate 2026-07-15)
- **Severity:** medium
- **Evidence:** the `LLMClient` interface (`internal/autoexpand/llm.go:26-29`, single `Generate(ctx,prompt,maxTokens)`) has ONE impl — `*OpenAILLM`; no `NewLLMClientFromConfig` factory exists anywhere at `255061b` (grep=0), so an Anthropic (or any non-OpenAI) provider cannot be selected by config. R7 pluggability + R19 Anthropic support both block on this. G20's fix removes the `p.llm.(*OpenAILLM)` assertion (the coupling); R19 adds the provider.
- **Why it matters:** R19 (operator mandate) requires Anthropic's Messages API as a first-class provider for the G05 jury + G20 auto-growth; without the factory + an `AnthropicLLM`, "supports Anthropic" is unmet.
- **DECISION:** add an `AnthropicLLM` (thin `net/http`, `POST {base}/v1/messages`, `x-api-key` + `anthropic-version: 2023-06-01`, NO temperature/top_p/top_k — newer Claude models 400 on non-default sampling; a policy refusal ⇒ error, never `("",nil)`) implementing `LLMClient`; add `NewLLMClientFromConfig(cfg,logger)` dispatching `openai|anthropic|local|helixllm` (fail-closed on unknown), mirroring `NewEmbedderFromConfig` (`internal/db/embedding.go:293`). Thin client, NOT the `anthropic-sdk-go` submodule (§11.4.28 house-style; avoids a G14-class dep escalation). **Alternatives rejected:** vendoring the full SDK for one endpoint; a verbatim `*OpenAILLM` port (would 400 on sampling params). Embeddings: Anthropic has NO first-party embeddings (§11.4.99-

verified live) — stays on G10's provider set; `"anthropic"` is never an `EmbeddingConfig.Provider`.
- **Test coverage:** 13 — 9 unit (factory dispatch ×4, header/request mapping incl. no-sampling-params, response parse, non-2xx error map, refusal handling), 2 integration (live `Generate` behind `integration` tag + SKIP-without-`ANTHROPIC_API_KEY`; `Pipeline.Run` via Anthropic asserts no placeholder), 2 paired §1.1 mutations. **Challenges:** yes.
- **STATUS (2026-07-16):** `Implemented (→ Fixed.md)` — `f083328`. Anthropic Messages API `LLMClient` provider + `NewLLMClientFromConfig` factory landed — closes G28's own scope (provider + factory); wiring into autoexpand/G03 is explicitly separate and NOT claimed done by this flip (supersedes the 2026-07-15 DESIGN DONE/Go-impl-PENDING note; design doc `research/r19_anthropic_api_support_design.md` preserved for history). **OPEN operator sub-decision (§11.4.66, non-blocking):** whether to ALSO expose an Anthropic-Messages-*shaped* server surface for R4 interop — R19's recommendation is NO (R4 already solved via MCP for every named CLI agent); recorded, deferred-safe default = do not build the redundant surface now (§11.4.101).

G29 — `Store.Search` advertises "hybrid vector search" but is trigram/ILIKE-only; `Store.VectorSearch` has zero callers

- **Category:** bug / doc-bluff (§11.4 / §11.4.6)
- **Severity:** high
- **Evidence:** `internal/skill/store.go:50-118` `Store.Search` — doc-comment claims hybrid vector search; body is ILIKE/trigram only, no query embedding used. `Store.VectorSearch` (`store.go:574-609`), the real pgvector KNN path, has **zero callers** (grep=0) across MCP `skill_search` + REST + pipeline dedup.
- **Impact:** the advertised semantic search is not delivered (keyword-only); a doc-comment claims a capability the code does not deliver (§11.4 code-layer bluff); the flagship pgvector search is dead (§11.4.124). R2/R13 make semantic retrieval core.
- **DECISION:** wire `VectorSearch` into `Search` (embed query → vector KNN + trigram, weighted/RRF merge) rather than correct-the-doc-to-keyword-only. **Alternatives rejected:** downgrading the doc-comment (abandons a core R2/R13 capability).
- **Test coverage:** unit (a semantically-near non-substring match ranks above a trigram-only match; `VectorSearch` reached), integration (live pgvector KNN), paired mutation (revert to ILIKE-only → hybrid test FAILs), regression. **Challenges:** yes.

- **STATUS (2026-07-15):** DISCOVERED (discovery-audit §11.4.118) → `research/p05_completion_audit_and_discovery.md`. Design + impl PENDING. HIGH — anti-bluff (doc claims a capability the code lacks).
- **STATUS (2026-07-17):** `Fixed (→ Fixed.md)` — hybrid search landed: `Store.Search` now embeds the query via `EmbedAsync` + runs `VectorSearch` KNN in parallel with trigram, merging results by weighted RRF. `Store.VectorSearch` now has real callers (MCP `skill_search` + REST). Doc-comment bluff corrected. Closes G29's own scope.

G30 — `learn_from_project` returns a job ID that can never be status-checked

- **Category:** bug / gap
- **Severity:** medium
- **Evidence:** `internal/skill/store.go:546-568` + `internal/mcp/tools.go:336` — the tool enqueues a job + returns a job ID, but no status-query path (no `GetJobStatus`-by-ID) exists; the caller cannot poll completion.
- **Impact:** R6 wizard + R14 real-time growth report a job ID the client cannot resolve → the progress-reporting contract is broken (§11.4.116-class).
- **DECISION:** add a job-status store (status by job ID: queued/running/done/failed + progress) + an MCP/REST status tool; the async pipeline writes status transitions.
- **Test coverage:** unit (status transitions), integration (enqueue→poll→done), paired mutation, regression.
- **STATUS (2026-07-15):** DISCOVERED → p05 doc. Design + impl PENDING.

G31 — `learn_from_project` `project_path` has zero validation → path-traversal / LFI primitive when G03 wiring lands

- **Category:** security (latent)
- **Severity:** high (latent)
- **Evidence:** `internal/mcp/tools.go:314-350` passes `project_path` unvalidated to `internal/codeanalysis/analyzer.go:196-240`, which walks the filesystem from it. Today the analyzer is not fully wired (G03) → latent; when G03 lands, an attacker-supplied `project_path` (`/etc`, `../../`, absolute host paths) becomes a local-file-inclusion / traversal read primitive on the (currently unauthenticated) MCP surface.

- **Impact:** arbitrary-directory read once G03 wires the analyzer → high.
- **DECISION:** validate `project_path` BEFORE the walk — canonicalize (`filepath.Abs+Clean`, resolve symlinks), enforce a config-driven allowlisted root prefix, reject traversal/absolute-escape, fail-closed. MUST land WITH or BEFORE G03.
- **Test coverage:** security (traversal `../`, symlink-escape, absolute-outside-root all rejected), unit (in-root accepted), paired mutation (drop validation → traversal test FAILs), regression.
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `cbaf5fb`. Path-traversal/LFI jail on `learn_from_project` `project_path` landed (`cbaf5fb feat(mvp/skill-graph): G31 path-traversal/LFI jail on learn_from_project project_path`, confirmed real — resolves the earlier hash-verification gap; supersedes the 2026-07-15 DISCOVERED note).

G32 — `registry.ReviewScheduler` fully built but has zero callers (dead flagship pipeline)

- **Category:** bug / dead-flagship (§11.4.108 layer-2/3, §11.4.124)
- **Severity:** high
- **Evidence:** `internal/registry` `ReviewScheduler` is complete but grep=0 callers — never instantiated/started by `cmd/server` or `cmd/worker`.
- **Impact:** the periodic skill-review / re-validation pipeline (a flagship maintenance mechanism) never runs in production → skills are never re-reviewed; SOURCE-green-but-RUNTIME-dead (§11.4.108).
- **DECISION (§11.4.124 investigate-before-remove):** git-history investigate whether it was wired-then-regressed vs never-completed; then WIRE it (start from `cmd/worker` under the single-owner advisory lock) + add the missing wiring tests — NOT remove (required functionality, not obsolescence).
- **Test coverage:** integration (scheduler runs a review cycle on a real DB), unit (cadence), paired mutation, regression + a §11.4.108 runtime-signature (scheduler tick observable on a clean deploy).
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `25516a5`. `registryReviewWorker` → `RunReviewOnce` wire-in landed + age-mark-ordering fix; supersedes the 2026-07-15 DISCOVERED note.

G33 — `Store.ExportToTOML` swallows a DB error → empty dependency name in exported skill file

- **Category:** bug

- **Severity:** low
- **Evidence:** `internal/skill/store.go` `ExportToTOML` discards a row-scan error path → a dependency name can serialize empty.
- **Impact:** a git-versioned TOML skill file (R14 source of truth) can be silently written with a blank dep name → corrupt round-trip.
- **DECISION:** propagate the scan error (fail the export) rather than emit a partial file. **Alternatives rejected:** best-effort partial export (corrupts the R14 SoT silently).
- **Test coverage:** unit (scan error → export errors, no partial file), regression, paired mutation.
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `b8d0e56`. `ExportToTOML` scan-error propagation landed; supersedes the 2026-07-15 DISCOVERED note.

G34 — unchecked `rid.(string)` type assertion in request-id middleware

- **Category:** weakness
- **Severity:** low-medium
- **Evidence:** `internal/api/middleware.go:184, 258-268` — `rid.(string)` without the comma-ok form; a non-string context value panics the request goroutine.
- **Impact:** a mis-set request-id context value → per-request panic (DoS-ish); today the setter is internal so unreachable, but a latent foot-gun.
- **DECISION:** comma-ok assertion with a safe fallback (empty/regenerated id); never panic on a context-value shape.
- **Test coverage:** unit (non-string context value → no panic, fallback id), regression, paired mutation.
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `08299e4`. `rid.(string)` comma-ok fix landed; supersedes the 2026-07-15 DISCOVERED note.

G35 — CLI + TUI send `Authorization: Bearer` but the server reads `X-API-Key` → both first-party clients 401 the moment G01 auth enforces

- **Category:** bug / integration regression (latent, §11.4.108)
- **Severity:** high
- **Evidence:** CLI `cmd/cli/commands/common.go:55` + TUI `.../api_client.go:76` set `Authorization: Bearer <key>`; the server auth middleware `internal/api/middleware.go:292` reads `X-API-Key`. Header mismatch → 401 for both clients once the G01 hardened auth is actually enforced on the live listener.

- **Impact:** the two shipped first-party clients cannot authenticate against their own backend the moment auth is enforced (the fix breaks the clients) — a real user-visible break.
- **DECISION:** unify the auth-header contract — fix both clients to send `X-API-Key` (server-canonical) OR make the server accept both (documented); add a contract test asserting client-sent header == server-read header (§11.4.135 recurrence guard).
- **Test coverage:** contract (client header == server header), integration (CLI/TUI authenticate against an auth-enforced server), paired mutation (revert a client to Bearer → contract test FAILs), regression.
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `6f334c1`. X-API-Key header unification across all 7 first-party senders landed; supersedes the 2026-07-15 DISCOVERED note.

G36 — SSRF blocklist: non-zero `0.0.0.0/8` hosts not explicitly blocked (residual; the dangerous `0.0.0.0` IS caught)

- **Category:** security (residual)
- **Severity:** low
- **Evidence:** G01/G02 Fable-xhigh review residual — `internal/validation/sandbox.go` blocks `0.0.0.0` itself (the localhost-mapping danger) + `additionalBlockedRanges`; other `0.0.0.0/8` addresses (RFC 1122 "this network") are not explicitly listed. Proven NOT live-reachable as a localhost bypass (only `0.0.0.0` maps to localhost on Linux).
- **Impact:** minimal — the exploitable case (`0.0.0.0`→localhost) is blocked; residual is defense-in-depth completeness.
- **DECISION:** add `0.0.0.0/8` to `additionalBlockedRanges` for completeness (never a legitimate egress target); low priority.
- **Test coverage:** unit (`0.0.0.x` rejected), paired mutation, regression.
- **STATUS (2026-07-16):** `Fixed (→ Fixed.md)` — `912cfb7`. `0.0.0.0/8` added to the SSRF blocklist; supersedes the 2026-07-15 TRACKED note.

G37 — Import-skills path honors client status on the proven-DEAD `api.Server` router (O3 consolidation)

- **Category:** weakness (latent / dead-path, §11.4.108)
- **Severity:** low
- **Evidence:** G02 Fable review residual — `internal/api/skills_handler.go:546-559` `handleImportSkills`/`createReqToModel`

correctly honors client status, but on the `internal/api.Server` router that is proven DEAD (the live server is the consolidated hardened listener; O3 tracks consolidating/removing the dead `api.Server`).
- **Impact:** none live today (dead router); relevant only if `api.Server` is re-activated. Tracked to avoid a §11.4.108 SOURCE-green-RUNTIME-dead confusion.
- **DECISION:** fold into the O3 dead-`api.Server` consolidation (§11.4.124 investigate-before-remove) — carry the correct status-handling into the live path or remove with the router.
- **Test coverage:** covered by the O3 consolidation tests; regression.
- **STATUS (2026-07-15):** TRACKED (G02 Fable review residual). Deferred to O3 (§11.4.101).

G38 — §11.4.208 request-history auto-capture hook not yet wired (ledger is reconstruction-only)

- **Category:** task / infra (§11.4.208(D); R24)
- **Severity:** low
- **Evidence:** `requests/history.md` (§11.4.208 operator-request ledger) created this session but populated by RECONSTRUCTION (§11.4.208(B)); no `UserPromptSubmit`-class hook appends a row at accept-time.
- **Impact:** future operator prompts are not auto-captured → relies on manual/reconstruction append (the exact loss risk R24 targets).
- **DECISION:** wire a `UserPromptSubmit`-class hook (or equivalent) that appends a newest-first row per new prompt with deterministic Track/alias derivation (§11.4.182) + honest `?`/UNKNOWN; project-local, decoupled (§11.4.28/§11.4.177).
- **Test coverage:** hook test (a simulated prompt appends exactly one correctly-shaped row), paired mutation (hook stripped → gate FAILs), regression.
- **STATUS (2026-07-16):** `Completed (→ Fixed.md)` — `0438d7e`. UserPromptSubmit hook wired (`.claude/settings.json` + `scripts/append_request_history.sh`, selftest 7/7) — same commit cited for `G41`; supersedes the 2026-07-15 FILED/Wiring-PENDING note. Residual: UNKNOWN: the captured rows' Model+effort field cannot be observed from the `UserPromptSubmit` event alone — tracked as a known limitation, not re-opened as a separate item.

Constitutional-compliance violations (R23 audit, 2026-07-15) — G39–G49

Source: `research/constitution_compliance_audit.md` (independent R23 audit — 230 anchors enumerated: **47 COMPLIANT · 34 VIOLATION · 26 PENDING-AT-COMPLETION · 108 N-A · 15 UNCONFIRMED**). Full per-anchor evidence lives in that doc; this register tracks the actionable violations. Audit-internal ids reconciled to **G39+** (audit's `G50` "no request-history doc" is **SUPERSEDED** — `requests/history.md` created this session; its remaining auto-capture-hook facet = **G38**).

§11.4.201 gate-methodology note (NOT project violations): the 10 `CM-COVENANT-114-NNN-PROPAGATION` gate "failures" are FALSE POSITIVES — `helix_skills` is a thin consumer using the sanctioned `@import constitution/CLAUDE.md` pointer form (§11.4.35), which recursively inherits every anchor, but the gates `grep` for literal anchor copies; the gates also default their scan root to `..` and swept 1167 unrelated sibling-project hits until scoped with `CONSUMER_ROOT`. These are §11.4.201 guard-asserts-real-condition issues in the gate SCRIPTS (constitution-submodule scope), not project non-compliance. `CM-CONTINUUM-RESUME-ENGINE-PRESENT` FAIL is likewise an engine-internal decoupling issue out of this project's scope. `CM-REPORTING-DIRECTIVES` PASS (real run).

G39 — Rootful container runtime default in Makefile

- **Type:** Bug. **Severity:** HIGH. **Status:** `Completed (→ Fixed.md)` — `b597623`.
- **Evidence:** `project/Makefile:36` `CONTAINER_RUNTIME ?= docker` (rootful default) — MUST default rootless (podman).
- **Constitution references:** §11.4.161, §11.4.76, §11.4.173.

G40 — No SQLite workable_items.db single-source-of-truth

- **Type:** Task. **Severity:** HIGH. **Status:** IN PROGRESS — Phase 1+2+3 DONE, Phase 4+5 PENDING.
- **Evidence:** `docs/workable_items.db` created with 115 items (90 G + 25 R). Phase 1: G01-G49 imported (18 Fixed, 70 Queued, 1 In progress, 1 Operator-blocked). Phase 2: R01-R25 imported (all Queued). Phase 3: bidirectional round-trip proven — `db-to-md` exports 115 items with dot-format headings, `md-to-db` re-imports all 115, `validate` OK. Engine fix: `renderItemBody` uses dot separator for

non-canonical IDs (Gxx, Rxx) so parser shape-1 regex matches on re-import. G45 (closed-vocabulary) resolved by import. G47 (id scheme) resolved by Option (a) — literal Gxx/Rxx as atm_id. Phase 4 (generators read from DB) + Phase 5 (deprecate direct markdown edits) pending.
- **Constitution references:** §11.4.93, §11.4.95, §11.4.74, §11.4.28, §11.4.108, §11.4.6.

G41 — Zero hooks wired in settings.local.json

- **Type:** Task. **Severity:** HIGH. **Status:** `Completed (→ Fixed.md)` — `0438d7e`.
- **Evidence:** `.claude/settings.local.json` wires ZERO hooks — the `guard-forbidden-commands.sh` PreToolUse guard (+ §11.4.182/ §11.4.191 guards) not installed.
- **Constitution references:** §11.4.109.

G42 — Insufficient test coverage

- **Type:** Task. **Severity:** HIGH. **Status:** FILED; PENDING (phased with impl).
- **Evidence:** only ~10 unit-ish tests; no stress/chaos/e2e/security-suite/ Challenges/HelixQA coverage yet (expands G04). Lands per-package as the Go spine proceeds.
- **Constitution references:** §11.4.27, §11.4.52, §11.4.85, §11.4.169.

G43 — No HTML/PDF doc exports; Docs Chain unwired

- **Type:** Task. **Severity:** HIGH. **Status:** IN PROGRESS — export pipeline LANDED (2026-07-18); Docs Chain wiring PENDING.
- **Evidence:** `scripts/export_docs.sh` generates HTML+PDF from 5 tracked docs via pandoc+weasyprint → `docs/exports/`. 10 files generated, 0 failures. Docs Chain declared in `helix-deps.yaml` but not yet invoked (separate follow-up).
- **Constitution references:** §11.4.12, §11.4.65, §11.4.106.

G44 — Missing revision headers on most docs

- **Type:** Task. **Severity:** MEDIUM. **Status:** FILED; backfill PENDING.
- **Evidence:** only `CONTINUATION.md` + `requests/history.md` carry the revision header; `REQUIREMENTS.md`/`IMPLEMENTATION_PLAN.md`/`SPEC.md`/ this register/all research docs lack it.

- **Constitution references:** §11.4.44.

G45 — Status/type/reopens closed-vocabulary not enforced

- **Type:** Task. **Severity:** MEDIUM. **Status:** FILED; RESOLVED by G40 adoption plan Phase 1 (closed-vocabulary enforced at import time via `workable-items validate`). Closes when G40 Phase 1 lands.
- **Evidence:** status/type/reopens closed-vocabulary not applied to the G0x findings; G40 adoption plan's Phase 1 import applies the closed set mechanically.
- **Constitution references:** §11.4.15, §11.4.16, §11.4.34.

G46 — Shell scripts lack companion docs

- **Type:** Task. **Severity:** LOW. **Status:** `Completed (→ Fixed.md)` — `97ce030`.
- **Evidence:** shell scripts under `project/scripts/` lack the companion `docs/scripts/<name>.md`.
- **Constitution references:** §11.4.18.

G47 — Gxx id scheme vs ATM-NNN naming

- **Type:** Task. **Severity:** MEDIUM. **Status:** FILED; operator-decision PENDING.
- **Evidence:** project uses G0x/R0x/P0x ids cross-referenced across 20+ docs, not ATM-NNN; a destructive rename is riskier than (a) documenting G0x as an approved alias OR (b) minting parallel ATM-NNN without renaming.
- **Constitution references:** §11.4.54.

G48 — README lacks Tracked-Items doc-link section

- **Type:** Task. **Severity:** LOW. **Status:** `Completed (→ Fixed.md)` — `74a88d1`.
- **Evidence:** README lacks the `Tracked-Items` doc-link section.
- **Constitution references:** §11.4.57.

G49 — No QWEN.md/GEMINI.md mirrors

- **Type:** Task. **Severity:** LOW. **Status:** `Completed (→ Fixed.md)` — `4b5e78a`.

- **Evidence:** no `QWEN.md`/`GEMINI.md` mirrors of the project CLAUDE.md/AGENTS.md.
- **Constitution references:** §11.4.157.

The **R23 full re-run at project completion** (every anchor re-audited to zero real violations, CM-gates + a §11.4.32 project sweep wired) is the terminal compliance gate.

Additional findings (post-R23 audit)

G51 — Silent migration-state desync from psql error handling

- **Type:** Bug. **Severity:** MEDIUM. **Status:** `Fixed (→ Fixed.md)` — `0fee489`.
- **Evidence:** `scripts/migrate.sh` runs `psql` without `-v ON_ERROR_STOP=on` and discards its stderr (`2>/dev/null`); `psql` exits `0` even when an individual SQL statement inside a migration errors.
- **Constitution references:** §11.4.201.

G52 — Health-probe repoint + ARCHITECTURE.md doc-drift

- **Type:** Bug. **Severity:** LOW. **Status:** `Fixed (→ Fixed.md)` — `7b9f40a`.
- **Evidence:** Health-probe repoint `/api/v1/health` → `/health` (TUI + CLI) + `ARCHITECTURE.md` doc-drift closed inline.

G53 — WaitForVectorIndexReady catalog-query correction

- **Type:** Bug. **Severity:** LOW. **Status:** `Fixed (→ Fixed.md)` — `707185a`.
- **Evidence:** `WaitForVectorIndexReady` catalog-query correction + hardened error handling.

G54 — gofmt drift in internal/validation/pipeline.go

- **Type:** Task. **Severity:** LOW. **Status:** `Fixed (→ Fixed.md)` — project-wide gofmt-clean.
- **Evidence:** `internal/validation/pipeline.go` carries pre-existing `gofmt` drift (~lines 66-76). G62's fix resolved this file's drift.

G55 — Phantom OpenAPI/doc-listed routes unimplemented

- **Type:** Bug. **Severity:** MEDIUM. **Status:** Queued.
- **Evidence:** Phantom OpenAPI/doc-listed routes beyond the `G52`-fixed `/api/v1/health` (e.g. `/api/v1/graph`, `/skills/:id/{evidence,validate}`) are documented/listed but unimplemented. Composes with `G09`.

G56 — docker-compose deployment contract broken

- **Type:** Bug. **Severity:** MEDIUM. **Status:** Queued.
- **Evidence:** `docker-compose` app-profile deployment contract is broken: double `ENTRYPOINT`/`command` argv-stacking defeats `--config`; the worker container re-runs the server binary. Composes with `G13`.

G57 — MCP ACPAdapter stdio transport was unwired

- **Type:** Bug. **Severity:** HIGH. **Status:** `CLOSED (commit 8fa4e27)`.
- **Evidence:** `internal/mcp.ACPAdapter` — the `--mcp acp` stdio transport was unwired; fixed by wiring the transport in + an idempotent-`Stop` fix.
- **Constitution references:** §11.4.108, §11.4.124.

Adjudication of the 8 mandated open items

#	Item	Finding	One-line decision
1	Embedding dimension (768/1536/384)	**G10**	Pin **768** default; template `vector(N)` from config; add startup model-dim==column-dim assertion + OpenAI length check; extend providers.
2	§11.4.28C single-canonical vs operator parent-priority+both-synced	**G14**	Parent copy = the one *logical* canonical; `submodules/<name>` = read-only mechanical mirror pinned by `sync_submodules.sh`; escalate this framing for sign-off before `--apply`.
3	TOON: no Go codec	**G08**	Implement/vendor a spec-conformant TOON codec with

#	Item	Finding	One-line decision
			golden vectors before advertising `application/toon`; do not claim TOON until it exists; revise openapi.yaml in lockstep.
4	Zero automated tests vs R8 ~100%	**G04**	Bootstrap `go test`, then per-package tables + paired mutations, security middleware + DAG first; coverage gate in CI.
5	Security behavioural proof missing	**G01** + **G04**	The CORS/api_key fixes aren't even wired (G01); wire the hardened server, then prove behaviour with security tests + paired mutations.
6	Aurora/ HarmonyOS feasibility	**G15**	Run the Flutter-OHOS + omprussia spike early; risk-flag with the exact blocker (§11.4.112), never bluff a build; freeze the backend contract first.
7	Silent-failure / TODO / stub patterns	**G02, G03, G11, G12, G20**	Cited: `treesitter.go:130,160,230,235,240`; `runner.go:317,321,332,347`; `skills_handler.go:552`; `pipeline.go:431`; sandbox process-fallback. Fix per each finding.
8	Rival docker-compose copies	**G13**	Make `project/deploy/` the canonical ops home; delete/merge the root compose; scripts + systemd unit reference the one file.

Resolved / non-findings (verified, for the record)

- **DB-layer dedupe (P0.T1) is done:** `go.mod` shows only `pgx v5` + `pgvector-go`; the 5 stray ORMs (ent/gorm/bun/go-pg/sqlx) are gone.
- **`deploy/.env` is NOT a leak:** it is untracked and `.env` is git-ignored (`.gitignore:21`).
- **`config/config.toml` uses valid `#` comments** (only the SPEC §8 *sample* is wrong — G19).
- **Vector columns and config default agree at 768** today (the risk is the absence of an *assertion*, not a current mismatch — G10).

- **Parameterised SQL throughout the `db` layer** (pgx `$1..$n`); the only string-built identifiers are table names, guarded by `sanitizeTableName` against a fixed caller set (residual foot-gun noted in G27, not an active injection).

Positive-evidence-only. Every "is/does" claim above is pinned to a file:line that was read during this audit; the two UNCONFIRMED sub-points (MCP `ImportFromTOML` edge fidelity in G07) are labelled as such and require a read of `internal/skill/import_export.go` to close.

Session-discovered + planned items (2026-07-16)

> All items in this section: `created_by=Claude`, `assigned_to=''`, external-tracker-push=SKIP(`tracker_client_absent` — no `.helix/reporting.yaml`-equivalent reporting config exists in this project, §11.4.202(4)/§11.4.10).

G58 — (Placeholder) Referenced in G136 severity assessment

- **Type:** TBD. **Severity:** MEDIUM (proposed per G136). **Status:** UNCONFIRMED.
- **Note:** G58 is listed in the Summary counts table as MEDIUM but has no individual entry in this register. Referenced alongside G55, G56, G60, G61, G64, G66 as "Bugs and gaps in ops-scripts, migrations, compose contracts, and search conflict-oracle." Conductor must investigate and file the actual finding.

New findings this round (G59–G68)

G59 — Embedding ingestion never wired; StoreSkillEmbedding is dead code

- **Type:** Bug. **Severity:** HIGH. **Status:** Fixed (2026-07-17) — `Store.Create` now calls `embedWriteThrough` (store.go:897-899) which invokes `db.StoreSkillEmbedding` (store.go:1031); `ClearSkillEmbedding` called on all failure/skip branches (store.go:1131); stale-vector-on-update handled via clearStaleEmbedding. G59's original evidence ("zero non-test

callers") is stale — the write-through path is fully wired with proper degradation posture.
- **Evidence:** `Store.Create`/`Store.Update` never write `skills.embedding`; the DB-layer embed function — confirmed as `db.StoreSkillEmbedding` at `internal/db/vector.go:180` — and `EmbedAsync` have zero non-test callers, so post-`G29` hybrid search degrades to keyword/trigram-only in practice.
- **Composes with:** G29, G10, G111.

G60 — Search conflict-oracle uses ranked Search instead of exact GetByName

- **Type:** Bug. **Severity:** MEDIUM. **Status:** Fixed (2026-07-17) — `pipeline.go:555,564` uses `GetByName` (exact `WHERE name = $1`), not `Search`. The comment at line 535-546 documents why: Search's RRF scoring makes limit=1 non-deterministic.
- **Evidence:** `internal/validation/pipeline.go`'s existence/conflict oracle uses ranked `Search(name,1)` instead of exact `Store.GetByName` — latent until `G59` AND `G29` fully land.
- **Composes with:** G29; sequenced to land WITH or AFTER G59.

G61 — Two divergent /health implementations

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued.
- **Evidence:** Two divergent `/health` implementations exist — the live inline handler in `cmd/server/main.go` vs the dead `internal/api.handleHealth` — with differing response body schemas.
- **Composes with:** G01 (O3 sub-scope), G09, G96. The conductor must resolve G61/G96/G01-O3 as ONE piece of work (§11.4.186 anti-divergence).

G62 — 20 files with gofmt drift project-wide

- **Type:** Task. **Severity:** LOW. **Status:** Fixed (2026-07-17) — `gofmt -w` applied to all 13 drifted files; `gofmt -l` returns empty; build/vet/test all green post-fix.
- **Evidence:** 20 files project-wide carry pre-existing `gofmt` drift. Fix scope is 18 standalone-hygiene files (minus `embedding.go` and `pipeline.go` tracked under G29/G54).

G63 — 4th divergent route-contract surface (registry CLI/TUI)

- **Type:** Bug. **Severity:** HIGH. **Status:** Operator-blocked.
- **Evidence:** A 4th divergent route-contract surface exists: the `registry` CLI/TUI command group is 100% unreachable; the TUI health indicator is permanently disconnected; live handlers ignore query parameters.
- **Composes with:** G01, G09, G61, G96.
- **Operator-Block-Details:** 5 product/ownership decisions (D1-D5) must be made before this item is implementable.
  - **Operator-Block-Details:**
    - **WHAT:** 5 product/ownership decisions (D1-D5) must be made before this item is implementable: D1 — whether the `registry` CLI/TUI command group is kept, rewired, or removed; D2 — which of the 4 divergent route-contract surfaces (live server / dead `internal/api.Server` / OpenAPI / `registry` group) becomes canonical; D3 — whether the TUI health indicator is reconnected to the live health endpoint or removed; D4 — whether live handlers should start honoring the query parameters they currently ignore, and which semantics; D5 — how this item's resolution sequences against `G61`/`G96`/`G01`-O3 (one merged work item vs three).
    - **WHY (self-resolution exhausted per §11.4.21):** (a) no CLI/ADB/SSH/API access gap — the code is fully readable, this is a product-scope decision not an access problem; (b) subagent delegation cannot substitute for an operator product decision; (c) existing repo tooling has no canonical-router config to consult; (d) no synthetic/mocked fallback resolves an ownership decision; (e) external research (§11.4.8) was performed for the surrounding technical patterns but cannot substitute for the operator's product intent.
    - **UNBLOCK CONDITION:** operator supplies explicit D1-D5 decisions (or delegates them to the conductor with an explicit scope).
    - **WHO:** operator (product/architecture owner).
  - **Design reference:** `scratchpad/new1_route_contract_design.md` (agent-local scratch path, cited by the source draft). **§11.4.197 follow-up:** this design doc is NOT yet relocated into the project's `research/` tree — it must be moved from scratchpad into an in-repo `research/` location (or re-derived if the scratchpad copy is unavailable) before the full route-contract decision can be

finalized and closed; tracked here as an open follow-up, not resolved by this edit.
    - **STATUS:** Operator-blocked. ### G64 — Unconditional schema_migrations row delete on down without .down.sql
- **Type:** Bug. **Severity:** MEDIUM. **Status:** `Fixed (→ Fixed.md).`
- **Evidence:** `scripts/migrate.sh:179-183` (the `else` branch) deletes the `schema_migrations` row on `down` even when there is no matching `.down.sql` file, producing a tracked-version desync. Composes with landed G51.

G65 — stop.sh rejects –compose flag

- **Type:** Bug. **Severity:** LOW. **Status:** `Fixed (→ Fixed.md).`
- **Evidence:** `scripts/stop.sh` only supports `--quiet`/`-q`/`-h`; `--compose` hits "unknown arg" (exit 2); `restore.sh:292`'s `2>/dev/null||true` swallows that failure. Composes with landed G13.

G66 — Seed corpus missing 3 prerequisite TOMLs

- **Type:** Bug. **Severity:** MEDIUM. **Status:** Queued.
- **Evidence:** Seed corpus is missing 3 prerequisite TOMLs; 4 of 8 seed files fail-closed on a clean import. Affects the `seed/` corpus.

G67 — qa-results gitignore vs curated QA evidence policy conflict

- **Type:** Task. **Severity:** LOW. **Status:** Operator-blocked.
- **Evidence:** `project/qa-results/` is not fully gitignored today (only `*.log` is ignored); a genuine policy conflict exists between gitignore-whole-dir (§11.4.30) and curate-only-at-release-prep (§11.4.83).
- **Operator-Block-Details:** decide the `project/qa-results/` tracking policy.
- **Constitution references:** §11.4.30, §11.4.83.
    - **Operator-Block-Details:**
        - **WHAT:** decide the `project/qa-results/` tracking policy — either (a) gitignore the whole directory (satisfies §11.4.30 hygiene, but risks losing curated QA evidence that §11.4.83 requires to stay committed), or (b) keep it tracked but curate-only-at-release-prep (satisfies §11.4.83, but requires disciplined pruning of raw/uncurated output to avoid §11.4.30 violations), or (c) some hybrid (e.g. gitignore raw subpaths, track only a `curated/` subdirectory).

        - **WHY (self-resolution exhausted per §11.4.21):** this is a direct conflict between two constitutional mandates (§11.4.30 vs §11.4.83) applied to the same directory; neither mandate subordinates the other, so the choice of resolution shape is a policy decision, not a technical one self-resolvable by the agent.
        - **UNBLOCK CONDITION:** operator picks the policy (whole-dir-gitignore / curate-only / hybrid).
        - **WHO:** operator.
    - **STATUS:** Operator-blocked. ### G68 — RemoveDependency coarse delete should be relation-type-aware
- **Type:** Bug. **Severity:** LOW. **Status:** Queued — VERIFY.
- **Evidence:** `RemoveDependency`'s coarse delete (removes ALL typed edges for a skill pair) should become relation-type-aware. `G25` fixed a DIFFERENT defect on the same function. Conductor MUST re-check `internal/skill/graph.go`'s current `RemoveDependency` body before filing/closing.
- **FACT (2026-07-16):** `RemoveDependency` is DEFINED at `internal/skill/graph.go:125` but is UNWIRED — its only caller is its own test. Dead per §11.4.124.
- **Composes with:** G25, G07.

Planned feature epics (G69–G123)

G69 — FEAT: GitHub Skills Source Ingestion & Sync

- **Type:** Feature. **Status:** Queued. **Severity (proposed, carried forward verbatim from the source draft's own stated severity — not independently reassessed here):** high (net-new capability, operator-mandated 2026-07-16).
- Add GitHub-repository skill-source registration + fetch/parse/import + non-destructive enhancement-delta application + regular re-sync, exposed via CLI + REST + MCP + TUI.
- Full detail pointers: `gh_skills_research/CATALOG.md` (12-repo research corpus), `gh_skills_research/DESIGN.md` (architecture), `gh_skills_research/WIRING_PLAN.md` (exact file:line wiring), `gh_skills_research/TRACKED_ITEMS.md` (the 23 sub-items below).
- Depends on `G06`/`G07` (DAG correctness, both landed `186e047`/`073192f`) — confirmed satisfied. `G80` and `G86` (sub-items touching the `internal/skill` package / `internal/mcp/server.go` respectively) MUST serialize behind the in-flight `G29` lane (§11.4.119 single-resource-owner + §11.4.191 work-to-track binding). `G123` (below) must resolve BEFORE `G70`'s migration (`004_skill_sources`) lands,

since G95 (the G93 umbrella's own schema item) independently claims the SAME migration number.

G70 — Migration: skill_sources + skill_source_mappings + skills.origin

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G69; conflicts with G95 pending G123.

G71 — Migration: skill_enhancement_proposals

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G70.

G72 — SourceSyncConfig + env overrides + config.toml example

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G69; overlaps G94 pending G123.

G73 — New AuditEvent constants for skill-source events

- **Type:** Task. **Severity:** LOW. **Status:** Queued. **Depends on:** G69.

G74 — internal/skillsource package: source registry CRUD

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G70; overlaps G97 pending G123.

G75 — internal/source/github: hand-rolled REST fetch client

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G74.

G76 — internal/source/github: shallow-clone fallback

- **Type:** Feature. **Severity:** LOW. **Status:** Queued. **Depends on:** G75.

G77 — internal/source/skillmd: SKILL.md parser

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G69.

G78 — internal/source/mapper: ParsedSkill → models.Skill + license gate

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G77.

G79 — internal/source/dedup: NEW/DUPLICATE/VARIANT classifier

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G78.

G80 — Store.ImportSkillModel (sibling to ImportFromTOML)

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G79; serializes behind the G29 lane.

G81 — internal/source/enhance: delta extraction + proposal store

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G71, G79.

G82 — internal/source/sync: per-source scan orchestrator

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G75, G76, G80, G81.

G83 — Worker wiring: JobTypeSourceRescan + sourceRescanWorker

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G82.

G84 — REST wiring: cmd/server/skillsource_routes.go + buildRouter

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G83.

G85 — CLI wiring: cmd/cli/commands/source.go

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G84.

G86 — MCP wiring: internal/mcp/source_tools.go

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G84; serializes behind the G29 lane.

G87 — TUI wiring: cmd/tui/sources.go (read-only)

- **Type:** Feature. **Severity:** LOW. **Status:** Queued. **Depends on:** G84.

G88 — e2e test: real anthropics/skills pipeline run

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G85, G86, G87.

G89 — Stress + chaos test suite for ingestion pipeline

- **Type:** Task. **Severity:** LOW. **Status:** Queued. **Depends on:** G88.

G90 — Vendor Challenges + HelixQA constitution submodules

- **Type:** Task. **Severity:** LOW. **Status:** Queued. **Depends on:** G69.

G91 — HelixQA Challenge bank entry for skill-source ingestion

- **Type:** Task. **Severity:** LOW. **Status:** Queued. **Depends on:** G89, G90.

G92 — Docs: README/API/CLI reference sync for new surfaces

- **Type:** Task. **Severity:** LOW. **Status:** Queued. **Depends on:** G91.

G93 — FEAT: Unified Multi-Source Skill Ingestion Subsystem

- **Type:** Feature. **Status:** Queued. **Severity (proposed, carried forward verbatim from the source draft's own stated severity — not independently reassessed here):** high (net-new capability, operator-mandated 2026-07-16).
- Add a pluggable multi-source skill-ingestion subsystem (filesystem real-time-watch/web/API/PDF/FTP/SMB/WebDAV) with a 7-stage extract → normalize → refine → dedup → create/wire pipeline, exposed via CLI + REST + MCP.
- Full detail pointers: `skill_ingestion_research/CODEBASE_MAP.md` (integration-point map), `skill_ingestion_research/RESEARCH.md` (library research: `fsnotify`, `goquery`, `go-readability`, `html-to-markdown`, `ledongthuc/pdf`, `kin-openapi`, `jlaffaye/ftp`, `go-smb2`, `gowebdav`), `skill_ingestion_research/DESIGN.md` (architecture, 7 honest boundaries), `skill_ingestion_research/TRACKED_ITEMS.md` (the 25 non-deferred + 4 deferred sub-items below).
- `G96` (router-duplication fix) — UNCONFIRMED: duplicates `G01`'s O3 sub-scope + `G61` above — conductor resolves as ONE item before any of `G94`-`G122` that assume "one canonical router" (namely `G115`) lands. `G111` (CREATE/EXTEND stage) should soft-serialize behind `G59` (embedding-population fix). Honest gaps already stated BY the source draft itself (never silently resolved here): no production-ready pure-Go NFS client exists (v1 = reuse the filesystem `Source` against an operator-mounted NFS export, `G105`); scanned/image-only PDF OCR has no clean permissively-licensed pure-Go path (`gen2brain/go-fitz` is AGPL-3.0 — an explicit operator license decision, not resolved here); only the filesystem source gets genuine real-time behaviour in v1 (the other four are one-shot bulk + deferred polling, `G119`).

G94 — Add config.IngestionConfig section

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G93; overlaps G72 pending G123.

G95 — Ingestion schema migration (004_ingestion.up/down.sql)

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G93; migration-number collision with G70, see G123.

G96 — Resolve internal/api.Server vs cmd/server/main.go router duplication

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G93; UNCONFIRMED: duplicates G01-O3 + G61, see G123.

G97 — Source interface + ItemRef/RawItem types

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G93.

G98 — Filesystem Source (bulk one-shot)

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G97.

G99 — HTTP/website Source (single URL + bounded crawl)

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G97.

G100 — PDF Source (upload-based)

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G97.

G101 — OpenAPI/API-schema Source

- **Type:** Feature. **Severity:** LOW. **Status:** Queued. **Depends on:** G97.

G102 — FTP Source

- **Type:** Feature. **Severity:** LOW. **Status:** Queued. **Depends on:** G97.

G103 — SMB Source

- **Type:** Feature. **Severity:** LOW. **Status:** Queued. **Depends on:** G97.

G104 — WebDAV Source

- **Type:** Feature. **Severity:** LOW. **Status:** Queued. **Depends on:** G97.

G105 — NFS honest-gap documentation + mount-based workaround

- **Type:** Task. **Severity:** LOW. **Status:** Queued. **Depends on:** G98.

G106 — HTML EXTRACT+NORMALIZE stage

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G99.

G107 — PDF EXTRACT+NORMALIZE stage

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G100.

G108 — OpenAPI EXTRACT+NORMALIZE stage

- **Type:** Feature. **Severity:** LOW. **Status:** Queued. **Depends on:** G101.

G109 — LLM-REFINE stage (interface-only, provider-agnostic)

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G106, G107, G108.

G110 — DEDUP stage

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G109.

G111 — CREATE/EXTEND + WIRE GRAPH RELATIONS stage

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G110; soft-serializes behind G59.

G112 — Ingestion job orchestration (durable)

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G111.

G113 — Recursive directory watcher (fsnotify + debounce)

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G98.

G114 — worker.JobTypeIngestSource + real handler

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G112.

G115 — REST /api/v1/ingest/* endpoints

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G114; assumes G96 resolved.

G116 — CLI ingest command group

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G115.

G117 — MCP skill_ingest_source tool

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G115.

G118 — Full anti-bluff test-suite + HelixQA Challenge bank

- **Type:** Task. **Severity:** LOW. **Status:** Queued. **Depends on:** G116, G117.

G119 — Periodic polling for FTP/SMB/WebDAV/API sources (deferred)

- **Type:** Feature. **Severity:** LOW. **Status:** Queued. **Depends on:** G102, G103, G104, G101.

G120 — Deep-research-extend stage activation (deferred)

- **Type:** Feature. **Severity:** LOW. **Status:** Queued. **Depends on:** G109.

G121 — TUI ingestion pane (deferred)

- **Type:** Feature. **Severity:** LOW. **Status:** Queued. **Depends on:** G115.

G122 — Source-removal → Skill staleness/deletion policy (deferred)

- **Type:** Feature. **Severity:** LOW. **Status:** Queued. **Depends on:** G118.

G123 — Architectural-overlap reconciliation (G69 vs G93)

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued.
- **Evidence:** Reconcile the overlapping schema/config/registry design between `G69` (GitHub-Skills-Ingestion) and `G93` (Unified-Multi-Source-Skill-Ingestion) before either's sub-items land.
- **Evidence (found independently this session — neither background research stream could see the other's output while running in parallel):**
  1. **Migration-number collision:** `G70`'s DDL claims `migrations/004_skill_sources.up.sql` as "next free"; `G95`'s DDL independently claims `migrations/004_ingestion.up.sql` as ALSO "next free". Both cannot be `004`.
  2. **Duplicate registry abstraction:** `G74` proposes a new `internal/skillsource` package with `Store` CRUD for a `skill_sources` table; `G97` proposes a new `internal/ingest/source.Source` interface + its own source-registry concept. Per §11.4.186 anti-divergence

this MUST be a single mechanism with adapter plugins, never duplicated per-source-type infrastructure.
3. **Duplicate config surface:** G72 (SourceSyncConfig) and G94 (IngestionConfig) both add a new top-level config section covering overlapping concerns (allowlisted roots/hosts, credential env-var naming, poll/scan cadence).

- **FACT (2026-07-16, source-confirmed) — reframes item 1 above:** NO migration-004 collision currently exists on disk — migrations/ holds only the 001/002/003 up/down pairs (git history for migrations/ = 3 commits only); neither G70 nor G95 has actually created a 004_* file yet, so item 1's "collision" is a PLANNING-time clash between two proposed-but-unimplemented DDL claims, not a present file collision. Separately confirmed a latent robustness gap in the migration runner itself: discoverMigrationsFS (internal/db/migrations.go:200-258) keys files into map[int64]string by numeric prefix and would SILENTLY let a lexicographically-later same-version file overwrite an earlier one with no error, if a real same-number collision were ever introduced — not presently triggered, but a genuine gap in the runner's collision handling. This item is therefore reframed from "a migration-004 collision exists" to "a latent silent-overwrite-on-collision gap in the migration runner, plus a planning-time number clash between G70 and G95 that must be resolved before either creates its 004_* file — no 004 exists yet".
- **Recommended resolution shape (a proposal for the conductor/operator to confirm, NOT decided here per §11.4.6):** treat G93's Source interface (G97) as the canonical pluggable-adapter abstraction, and re-scope G69's GitHub-specific sub-items (G74-G92) to implement ONE concrete Source adapter against that interface rather than a parallel internal/skillsource registry — collapsing G70+G95 into a single migration, and G72+G94 into a single config section. **This is a genuine architecture decision, not a mechanical fix — do not auto-apply this recommendation without conductor/operator sign-off.**
- **STATUS:** Queued.

G124 — FEAT: Auto-generated, always-in-sync Skills-tree documentation catalog (docs/skills/)

- **Type:** Feature. **Status:** Queued. **Created-by:** Operator. **Assigned-to:** Operator.
- **One-liner:** docs/skills always-in-sync structurally-organized Skills-tree catalog auto-generated from the skill store, exported (md/html/pdf), and auto-synchronized via Docs Chain + hooks + §11.4.86 roster fingerprint.

- **What:** maintain `docs/skills/` as a complete, structurally-organized (tree: index → category → per-skill detail) catalog of EVERY skill in the System with details + descriptions (name, kind, description, dependencies/6 relation types, resources), GENERATED from the DB/skill store, exported md+html+pdf (§11.4.65), and AUTOMATICALLY kept in sync via §11.4.106 Docs Chain context + §11.4.86 sha256 roster fingerprint + §11.4.109/§11.4.164 hooks, configurable/triggerable via CLI + REST + all clients.
- **Acceptance:** catalog present + tree-structured + one detail page per skill; regeneration re-armed by any skill add/modify/remove (fingerprint drift); md/html/pdf exports in sync; Docs Chain context registered; hook wires it out-of-the-box; four-layer coverage §11.4.4(b) + self-validated generator §11.4.107(10) + paired §1.1 + real-DB e2e §11.4.27; no bluff.
- **Composes with:** §11.4.12/.44/.60/.65/.86/.106/.109/.164/.171/.197/.202.
- **Note:** The G124 design doc is published at `research/g124_docs_skills_catalog_design.md` (this session). The scratchpad `scratchpad/docs_skills_catalog_research/` that originally fed it does not exist in the local tree (cleaned up after prior sessions); the committed research doc supersedes it. Sub-items G125–G135 are decomposed below.
- **Source-side advanceable now:** yes (generator design + skill-model mapping). **Build/DB-gated:** the live regeneration proof needs a running DB with seeded skills.
- **STATUS:** Queued.

G125 — Build the skillscatalog generator

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G124.
- **Description:** Build the `skillscatalog` generator: DB → deterministic Markdown tree + sha256 roster fingerprint sidecar (§11.4.86).

G126 — Wire docs skills-catalog generate|verify CLI subcommand

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G125.

G127 — REST handlers for catalog regenerate + status

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G125.

G128 — MCP skill_catalog_status tool

- **Type:** Feature. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G125.

G129 — Periodic reconciliation worker job for catalog

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G125.

G130 — Immediate-tick signal from skill-graph write paths

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G125, G129.

G131 — guard-skills-catalog-fresh.sh PreToolUse guard

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G125, G126.

G132 — Auto-propagation of catalog guard hook

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G131.

G133 — Docs Chain context for skills catalog

- **Type:** Task. **Severity:** MEDIUM. **Status:** Blocked. **Depends on:** G125.
- **Blocked:** not functionally completable until Docs Chain is incorporated (X1/P13.T1 clear).

G134 — Anti-bluff proof plan for catalog generator

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G125.

G135 — HelixQA Challenge bank entry for catalog

- **Type:** Task. **Severity:** MEDIUM. **Status:** Queued. **Depends on:** G125, G134.

G136 — Task: Retroactive severity assessment for session-discovered items G52–G137

- **Type:** Task. **Status:** Queued.
- **One-line description (§11.4.171, ≥6 words, subject+goal):** Assign evidence-based CRITICAL/HIGH/MEDIUM/LOW severity ratings to every session-discovered item `G52` through `G124` — and equally the `G124`-umbrella sub-items `G125`–`G135` plus this item `G136` itself, which inherit the same treatment so that NO session-discovered item is left un-assessed — using the same rubric already applied to `G01`-`G37`, and fold the results into the Summary counts table at the top of this file.
- **Evidence:** cited by the Summary-counts Note (top of this file) and by `G45`/`G54`/`G55`/`G56` and every other item marked “severity not independently assessed” — this id gives that recurring caveat a single tracked home instead of a bare unattributed clause.
- **Assessment result:** the per-item severity assessment has been published at `research/g136_severity_assessment.md` (this session). That doc is the canonical per-item rationale; the Summary counts table at the top of this file has been updated to reflect those proposals as the working baseline.
- **Depends on:** none blocking — can start at any time; individual items’ severities should be finalized as each one’s own investigation concludes.
- **STATUS:** Queued (preliminary assessment completed 2026-07-17 — see below).

Preliminary severity assessment table

Assessed 2026-07-17 per the same rubric as G01–G37. Items are grouped by outcome; full per-item rationale lives in `research/g136_severity_assessment.md`. Severities proposed here are the WORKING BASELINE — not final — and should be reviewed as each item lands or is re-investigated (§11.4.6).

HIGH (68 items)

Items	Rationale
G29, G31, G32, G35, G39–G43, G57, G59, G63, G137	Un-wired flagship pipelines, latent security holes, constitutional violations, or discovered contract-drift that blocks core functionality.
G69–G92 (×24), G93–G122 (×30)	Operator-mandated net-new feature epics (GitHub-ingestion and multi-source ingestion). Falls under §11.4.197 — unbuilt flagship risk if left un-resourced.

MEDIUM (35 items)

Items	Rationale
G55, G56, G58, G60, G61, G64, G66	Bugs and gaps in ops-scripts, migrations, compose contracts, and search conflict-oracle that degrade reliability but are not blocking.
G123	Architecture-overlap reconciliation — essential before G69/G93 sub-items land but a coordination task, not a defect.
G124–G135 (×12)	Docs catalog feature — net-new capability, not a regression; planned with honest gaps already declared.

LOW (18 items)

Items	Rationale
G52, G53	Both CLOSED; included for historical completeness.
G54, G62	`gofmt` drift — cosmetic, not behavioural.
G65, G67, G68	

Items	Rationale
	Ops-script edge cases, policy decisions, dead-code refinement. All deferred or low-impact.

N/A (1 item)

Items	Rationale
G136	This assessment task itself cannot receive a severity — it would be a self-referential rating (§11.4.6).

Summary table (all G01–G137)

See the revised Summary counts at the top of this file.

G137 — Bug: autoexpand gap-detection is inert against any graph the store API constructs

- **Type:** Bug. **Status:** Queued.
- **One-line description (§11.4.171, ≥6 words, subject+goal):** `internal/autoexpand/pipeline.go`'s `DetectGapsForSkill`/`collectGapsFromTree` (and the sibling `detectGapsForSingleSkill`) key gap detection on `dep.DependsOn == uuid.Nil`, a condition that is UNREACHABLE for any graph the store API constructs — so `Pipeline.Run` structurally returns `SkillsCreated=0` against every store-constructed graph, and the auto-growth pipeline is reachable-but-inert even after G03's worker-wiring lands.
- **Evidence (source-confirmed, §11.4.6; re-verified against the current tree, §11.4.199):** `skill_dependencies.skill_id`/`depends_on` are PRIMARY KEY columns (`migrations/001_initial.up.sql:25-29`: `PRIMARY KEY (skill_id, depends_on)`, both `REFERENCES skills(id) ON DELETE CASCADE`). **Corrected mechanism (2026-07-16, Fable-xhigh re-review finding — the ORIGINAL wording here was proven wrong and is replaced, not merely annotated):** PK membership does make the column implicitly NOT NULL, so a literal `NULL` in `depends_on` is rejected — but a *zero* UUID (`00000000-0000-0000-0000-000000000000`) is a non-null value, and on live pg16 it is rejected by the `skill_dependencies_depends_on_fkey` FK constraint instead, NOT by the NOT NULL check; a hand-inserted `skills` row with `id = '00000000-...'` is itself schema-valid

(the INSERT succeeds — no CHECK forbids the all-zero UUID), and once such a row exists a zero-UUID edge CAN persist. So "PK ⇒ NOT NULL ⇒ Postgres refuses any nil/zero `depends_on`" is FALSE as an unconditional schema guarantee, and the original title/description's "any real, schema-valid skill graph" over-scoped the claim (a hand-inserted zero-UUID skill row IS schema-valid). The TRUE backstop — matching this file's own Open-questions-ledger entry for "§11.4.197 follow-ups #4" below (whose own stale citations are corrected in this same pass) — is the FK constraint PLUS `Store.Create`'s nil-ID guard (`internal/skill/store.go:648-650`: `if skill.ID == uuid.Nil { skill.ID = uuid.New() }`), together with `AddDependency`'s both-endpoints-exist pre-check (`internal/skill/graph.go:22-53`), `ImportFromTOML`'s name-resolved-existing-IDs-only inserts (`internal/skill/import_export.go:112-138`), and `validation.Pipeline.CrossReference`'s independent `uuid.Nil` hard-error (`internal/validation/pipeline.go:551-552`): no `skills` row created by any store-API-driven path ever carries a nil or hand-picked-zero id, so the FK can never match one, so a zero-UUID edge can never persist — for any graph the store API itself builds. That scoped claim, not the unconditional one, is what this bug rests on. `Store.GetByName`'s dependency loader (`internal/skill/store.go:490`, mirrored at `:615` for `GetTree`) does a `JOIN skills ds ON sd.depends_on = ds.id` (an INNER JOIN, the SQL default) — every `models.SkillDependency.DependsOn` a caller observes via `GetByName`/`GetTree` is therefore, by construction, a real, existing skill id for any graph the store API built; it can never be `uuid.Nil` on that path. `collectGapsFromTree` (`internal/autoexpand/pipeline.go:145-170`) and `detectGapsForSingleSkill` (`pipeline.go:481-506`, doc comment at `:480`) both gate their gap-append on exactly `dep.DependsOn == uuid.Nil` (`pipeline.go:157` and `:489` respectively — CURRENT-TREE-VERIFIED line numbers; both citations were off by +12 lines in the prior revision of this entry, `:469-494`/`:477`, because this same fix round's F1 inserted ~12 comment lines earlier in the file, between `DraftSkill` and `detectGapsForSingleSkill` — `pipeline.go:157` itself did not shift) — a condition unreachable via either read path (`GetTree` or `GetByName`) for any graph the store API constructs. This is the SAME fact already source-confirmed in this file's own "Open-questions ledger" entry for "§11.4.197 follow-ups #4/#5" (nil-UUID FK edges) and independently documented, with the SAME now-corrected over-broad phrase, in the header comments of `internal/autoexpand/pipeline_crossreference_integration_test.go` and `internal/worker/autoexpand_integration_test.go` (both call it "a real, separate, out-of-scope defect in the gap-DETECTION half of this

package" / "a real, separate, out-of-scope finding", and both had their "any real, schema-valid skill graph" phrase corrected to "any graph the store API constructs" in this same pass) — this item is that finding's first tracked home; no prior Gxx id existed for it.
- **Why it matters:** even with G03's worker-loop wiring landed and F1 (this fix round) making `DraftSkill` provider-agnostic, `Pipeline.Run`'s own top-level gap scan can never find a gap to draft against on any graph a running system would actually construct — the auto-growth pipeline is wired end-to-end but permanently a no-op in production, silently. A feature that always reports `SkillsCreated=0` with no error is a §11.4/§107 PASS-bluff risk if ever presented as "auto-growth is enabled" without this caveat.
- **RESOLUTION:** Both (a) and (b) are now implemented together: (b) — `collectGapsFromTree` and `detectGapsForSingleSkill` use name-based `skillNameExists()` checks via the store instead of the unreachable `uuid.Nil` gate; (a) — `Run()` pre-collects registry-level gaps via `DetectGaps()` (name-based `GetMissingSkills`) and merges them into each skill's gap list at every depth layer, deduplicating by `MissingDepName`. Options (c) (schema reconciliation) and (d) (close per §11.4.90/§11.4.112) are documented-but-not-taken alternatives.
- **Composes with:** `G03` (this wiring round exposed the inertness as a concrete, testable fact rather than a theoretical one — see `pipeline_crossreference_integration_test.go` and its no-LLM sibling `pipeline_crossreference_test.go`, both of which had to construct their `Gap` manually instead of driving it through `Pipeline.Run`'s own scan, for exactly this reason; the worker package's `autoexpand_integration_test.go` demonstrates the SAME inertness the OTHER way — it drives `Pipeline.Run`'s own real scan (no manual `Gap{}`) against a seeded, store-API-constructed graph and honestly asserts `SkillsCreated == 0`), `G20` (auto-expand design/drafting — a sibling weakness in the SAME package, distinct facet: G20 is about WHAT gets drafted/persisted once a gap is found, G137 is about WHETHER a gap is ever found at all), `§11.4.6` (no-guessing — the unreachability is proven from schema + query text, not assumed), `§11.4.124` (this is a design/logic gap, not dead code — the functions ARE called by `Pipeline.Run`, they just can never match), `§11.4.197` (an un-tracked "third state" finding — this item is its resolution: the inertness was previously flagged only in test-file comments, never a tracked workable item).
- **Test coverage:** unit (construct a `models.SkillTreeNode`/`models.Skill` graph in-memory proving `collectGapsFromTree`'s condition is unreachable via any value `GetTree`/`GetByName` can actually produce), integration (real-DB: attempt to persist a `skill_dependencies` row with `depends_on = uuid.Nil` and confirm Postgres rejects it —

proving the NOT-NULL half; a second case attempting a ZERO UUID against an FK with no matching `skills` row, confirming the FK-violation half, distinct from the NOT-NULL half per the corrected mechanism above), mutation (a fix that switches detection to a reachable model must have a paired §1.1 mutation proving the OLD `uuid.Nil` check, if reintroduced, again returns zero gaps against a seeded real-name-mismatch graph). **Challenges:** yes (auto-growth-must-actually-detect-a-gap-on-a-real-graph). **HelixQA:** yes.

- **STATUS:** `Fixed (→ Fixed.md)` — `2026-07-17`. Fix implements RESOLUTION options (a)+(b): `collectGapsFromTree` and `detectGapsForSingleSkill` now use name-based `skillNameExists` checks against the store instead of the unreachable `dep.DependsOn == uuid.Nil` guard; `Run()` additionally pre-collects registry-level gaps via `DetectGaps()` (name-based `GetMissingSkills`, always reachable) and merges them into each skill's gap list at every depth layer, deduplicating by `MissingDepName`. `go build`/`go vet`/`gofmt` clean; full-suite `go test -race ./...` GREEN (24 pkgs). The *openness scope* of the original bug — that `Pipeline.Run` could never find a gap to draft against — is CLOSED. Residual (pre-existing, not re-opened): `handleValidate` and `handleCodeAnalysis` still stubs (G03); `runAutoExpandCycle` ticker still log-only (G03); `createMinimalDraft` placeholder-persist still active (G20). The RESOLUTION section's text above is updated to note that (a)+(b) are now implemented, leaving (c)+(d) as documented-but-not-taken alternatives.

Open-questions ledger — per-item resolution state (§11.4.6)

- **§11.4.197 follow-ups #4/#5** — RESOLVED (2026-07-16, source-confirmed), no new item filed:
  - **#4 (nil-UUID FK edges): FACT:** a nil/zero UUID edge CANNOT be silently persisted by any store-API-driven path. `skill_dependencies.skill_id`/`depends_on` are implicitly NOT NULL (PK columns, `migrations/001_initial.up.sql:26-29`, widened `002_granularity.up.sql:36-38`, FK `REFERENCES skills(id)`) — this blocks a literal `NULL`; a *zero* UUID is blocked separately, by the `skill_dependencies_depends_on_fkey` FK constraint, not the NOT NULL check (see `G137`'s corrected-mechanism note for the full distinction — a hand-inserted `skills` row with a zero-UUID id is itself schema-valid, so the FK is the operative backstop, not PK-implied NOT NULL alone). `AddDependency` (`internal/skill/graph.go:22-53`) pre-checks both endpoints; `ImportFromTOML` (`internal/skill/`

`import_export.go:112-138`) only inserts name-resolved existing IDs; `Store.Create` (`internal/skill/store.go:647-699`, dependency-insert loop at `:687-699`) relies on the FK; `CrossReference` (`internal/validation/pipeline.go:548-552`) hard-errors on `uuid.Nil`; every skill gets `uuid.New()` (`store.go:648-650`, corrected from this entry's earlier `:358-360` citation — current-tree-verified, 2026-07-16) so no nil-ID skill row exists → the FK is an effective backstop for any graph the store API itself builds. Test `graph_removedep_g25_test.go:136-151` proves the FK must be DROPPED to even construct a dangling edge.
    - **#5 (ExportToTOML direction): FACT:** no mismatch exists. `ImportFromTOML` (`import_export.go:22`) = TOML→DB (writes); `ExportToTOML` (`import_export.go:345`) = DB→TOML (reads via `GetByName`). Round-trip stability was fixed by `G07` (`import_export.go:367-373`), all 6 relation types emitted (`405-443`), proven by tests `g07_roundtrip_test.go` + `g33_export_empty_dep_name_test.go`.
    - Both follow-ups are CLOSED by this source-confirmation; citing `G07` (`073192f`) as the landing commit for #5's round-trip stability.
- **`G59` name discrepancy** — RESOLVED (2026-07-16, source-confirmed): the real symbol is `db.StoreSkillEmbedding` (`internal/db/vector.go:180`); `db.StoreEmbedding` was only ever a doc-comment misnaming at `vector.go:179`, never a second symbol. See the FACT line on `G59`'s own bullet above.
- **`G68` verify status** — the "coarse delete" facet is STILL genuinely unconfirmed against current source (see `G68`'s own `VERIFY` flag above); separately, this session source-confirmed a DIFFERENT fact — `RemoveDependency` is unwired/dead code (§11.4.124) — which does not resolve the coarse-delete question.
- **`G93`'s own already-declared honest gaps** (no viable pure-Go NFS client; AGPL-licensed OCR path only; non-filesystem sources get no real-time push) are carried forward verbatim from `skill_ingestion_research/DESIGN.md/RESEARCH.md` — not independently re-verified this session, not re-litigated.
- **`G73`/`G90`** (Challenges + HelixQA submodule vendoring) is flagged in its own source draft as a blocking dependency for `G91` — this plan preserves that blocking relationship but does not independently verify whether the two submodules are already vendored elsewhere in this project.